✎ complete study notes & cheat sheets

Docker & Kubernetes

Every topic, start → finish ↗

Colour-coded: defs · formulas · errors

Print it, scribble on it! ✨

The full **DevOps Directive** beginner → pro curriculum — containers, images, networking, storage, orchestration, Helm, security, CI/CD and cloud deployment — rewritten as colourful handwritten notes you can actually revise from.

26 Sections Covered | Worked Examples | Diagrams & Mind-Maps | Per-Chapter Cheat Sheets | Practice Q&A

by Idriss ZABARI

Compiled by Idriss ZABARI · From the DevOps Directive Docker & Kubernetes course · For study & revision

✎ Table of Contents

Handwritten notes for the complete DevOps Directive Docker & Kubernetes curriculum

🐋 PART 1 — Docker & Containers

1. History & Motivation — bare metal, VMs, containers, trade-offs 5
2. Technology Overview — cgroups, namespaces, union FS, Docker architecture 8
3. Installation & Setup — Docker Desktop vs Engine, first containers 12
4. Using 3rd-Party Containers — volumes, mounts, databases, CLI tools 15
5. Example Web Application — the 3-tier demo app (React + Node/Go + Postgres) 20
6. Building Container Images — Dockerfiles, layers, multi-stage, best practices 23
7. Container Registries — Docker Hub, GHCR, ECR, GCR, auth & credential helpers 30
8. Running Containers — docker run vs compose, key runtime flags 32
9. Container Security — image + runtime hardening, scanning, signing 36
10. Interacting with Docker Objects — images, containers, volumes, networks CLI 38
11. Development Workflows — hot reload, debuggers, tests, CI image pipeline 41
12. Deploying Containers — Railway, Docker Swarm & Kubernetes 44

☸ PART 2 — Kubernetes & Orchestration

1. History & Motivation — ops eras and the move to orchestrators 47
2. Technology Overview — nodes, control plane, data plane, system components 49
3. Installation & Setup — Devbox, KinD, Civo & GKE clusters 52

4.1 Namespaces, Pods, ReplicaSets & Deployments 54
4.2 Services — ClusterIP, NodePort & LoadBalancer 59
4.3 Ingress & the Gateway API 62
4.4 Jobs, CronJobs & DaemonSets 66
4.5 StatefulSets & Persistent Storage (PV / PVC / StorageClass) 69
4.6 ConfigMaps & Secrets 73
4.7 RBAC & Pod SecurityContext 76
5 Helm — charts, templating, 3rd-party installs, release lifecycle 79
6 Demo Application — the 3-tier app + Python load generator 83
7 Deploying the Demo Application — Deployments, Services, IngressRoutes, Jobs 85
8 Extending Kubernetes — CustomResourceDefinitions & Operators 90
9 Auxiliary Tooling — CloudNativePG, Trivy Operator, cert-manager 93
10 Developer Experience — inner loop, Tilt, secrets management 96
11 Debugging Kubernetes — the troubleshooting flow & kubectl toolkit 99
12 Deploying to Multiple Environments — Kustomize, Helm, Kluctl 102
13 Cluster & Node Upgrades — safe control-plane / node-pool upgrades 105
14 CI/CD for Kubernetes — image pipelines & GitOps 108

📑 PART 3 — Cheat Sheets

A Docker Cheat Sheet — every command & concept on one spread 111
B Kubernetes Cheat Sheet — kubectl, YAML anatomy & resources 115
C Master Cheat Sheet — the entire course condensed 119

Docker · Part 1

1 History & Motivation

Why containers exist — the journey from bare metal to VMs to containers, and the trade-offs between them.

📖 Definition — What is a container?

A **Docker container image** is a lightweight, standalone, executable package of software that includes everything needed to run an application: the code, runtime, libraries, environment variables and config files. At runtime the image becomes one or more **containers**.

1.1 The evolution of virtualization

Every era solved the previous era's pain point — then created a new one. Learn the story, remember the trade-offs.

Bare Metal — Everything runs directly on the **physical hardware**. One OS, apps share it all.

Virtual Machines — A **hypervisor** carves one host into many isolated virtual machines — each with its own OS.

Containers — Native Linux features (**cgroups + namespaces**) isolate processes that **share one kernel**.

🏭 Bare Metal

- OS + libs + apps on real hardware
- 😖 Hellish dependency conflicts
- 📉 Low utilization
- 💥 Large blast radius
- ⏱ Start/stop in **minutes**
- 🐢 Provision/decommission in **hours–days**

🧱 Virtual Machines

- Hypervisor slices host into VMs
- ✅ No dependency conflicts
- 📈 Better utilization
- 🎯 Small blast radius
- ⏱ Start/stop in **minutes**
- ⚡ Provision in **minutes**

📦 Containers

- Share the host kernel
- ✅ No dependency conflicts
- 📈 Even better utilization
- 🎯 Small blast radius
- ⚡ Start/stop in **seconds**
- 🚀 Provision in **seconds**

🧠 Memorable idea

VMs virtualize hardware; containers virtualize the operating system. That single sentence explains almost every difference in speed, size and isolation you'll ever meet.

⚠️ Common misconception

Containers are **not** "tiny VMs". They are just **processes** on the host kernel, fenced in by kernel features. Less isolation than a VM — which is why security matters (see Chapter 9).

1.2 The container timeline (and the Docker moment)

*Docker didn't invent containers — it made them **usable**. Developer experience drove mass adoption.*

💡 Exam tip

Be ready to explain **why containers replaced VMs for app deployment**: speed (seconds vs minutes), density (share kernel), portability (image = code + deps), and dev/prod parity. Then acknowledge the trade-off: **weaker isolation** than a hypervisor.

1.3 Trade-offs at a glance

Attribute	Bare Metal	Virtual Machine	Container
Isolation level	None (shared OS)	Strong (own kernel)	Process-level (shared kernel)
Resource overhead	Lowest (but wasteful)	High (guest OS per VM)	Lowest usable / dense
Startup time	Minutes	Minutes	Seconds (even ms)
Provisioning	Hours–days	Minutes	Seconds
Portability	Low	Medium	High (image format)
Good for dev?	No	Heavy	✔ Lightweight enough to develop in

🗒️ Scribble:

"Performance" on a chart hides nuance — a VM/container doesn't inherently lose much raw speed. What matters is **control**: storage locality, hardware accelerators, network proximity. Know this for the exam.

🌟 Key Takeaways — Chapter 1

- **Bare metal:** apps run directly on hardware. Simple, but conflict-prone, low density, big blast radius.
- **Virtual machines:** a hypervisor creates isolated VMs each with a full OS — better isolation and provisioning.
- **Containers:** share the host kernel and use cgroups + namespaces — fastest, densest, most portable, but less isolated.
- A container **image** packages code + runtime + libraries + config; a **container** is a running instance of it.

- Docker didn't invent containers (LXC did, 2008) — it won on **developer experience**.

✅ Quick-Review Checklist

- [ ] Define "container image" in one sentence
- [ ] List 3 pains of bare metal
- [ ] Explain the role of a hypervisor
- [ ] Name the 2 Linux features containers rely on
- [ ] Compare startup times: BM / VM / container
- [ ] State the main trade-off of containers vs VMs

📝 Practice Questions

Q1. **What is a container image?**
→ A lightweight, standalone, executable software package containing code, runtime, libraries, settings and config needed to run an app.

Q2. **What component lets one physical host run many VMs?**
→ The **hypervisor**.

Q3. **Which two Linux kernel features isolate and limit containers?**
→ **Namespaces** (isolation) and **cgroups** (resource limits).

Q4. **Why do containers start in seconds while VMs take minutes?**
→ Containers share the host kernel and don't boot a full guest OS — they just start a process.

Q5. **Give one downside of containers relative to VMs.**
→ Weaker isolation — they share the kernel, so a kernel exploit has a larger blast radius.

Q6. **Did Docker invent containers? Justify.**
→ No — LXC existed in 2008. Docker popularised containers by dramatically improving developer experience.

Q7. **What are the main benefits of a VM over bare metal?**
→ No dependency conflicts, better utilization, smaller blast radius, faster provisioning.

2 Technology Overview

The three Linux building blocks that make containers possible, plus the Docker component architecture.

Containers are not magic — they're a clever use of features that have been in the Linux kernel for years. Learn these three and you understand "how containers work".

2.1 The Linux building blocks

📊 cgroups

Control groups organise processes into hierarchical groups whose **resource usage** (CPU, memory, disk I/O) can be limited and monitored.

🧩 namespaces

A namespace **wraps a global system resource** so processes inside think they have their own isolated instance of it — invisible to everyone else.

🗂️ union filesystem

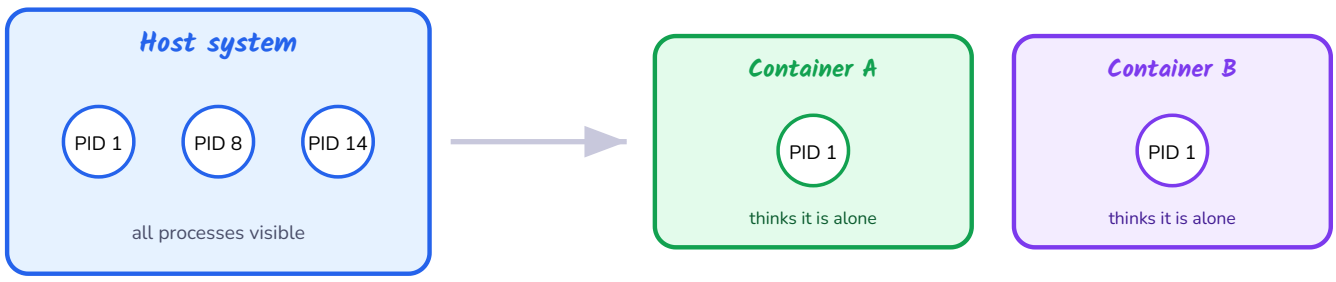
Overlays separate filesystems ("branches") into one **coherent merged filesystem**, so shared layers are stored only once.

Cgroups — "how much can I use?"

A container runtime uses cgroups to put limits on a container. Examples of what it can constrain:

Resource	cgroup knob	Meaning
CPU cycles	`cpu.shares`	Use up to XX% of CPU cycles
Memory	`memory.limit_in_bytes`	Use up to YY MB of memory
Block I/O	`blkio.throttle.read_bps_device`	Throttle reads to ZZ MB/s

Namespaces — "what can I see?"

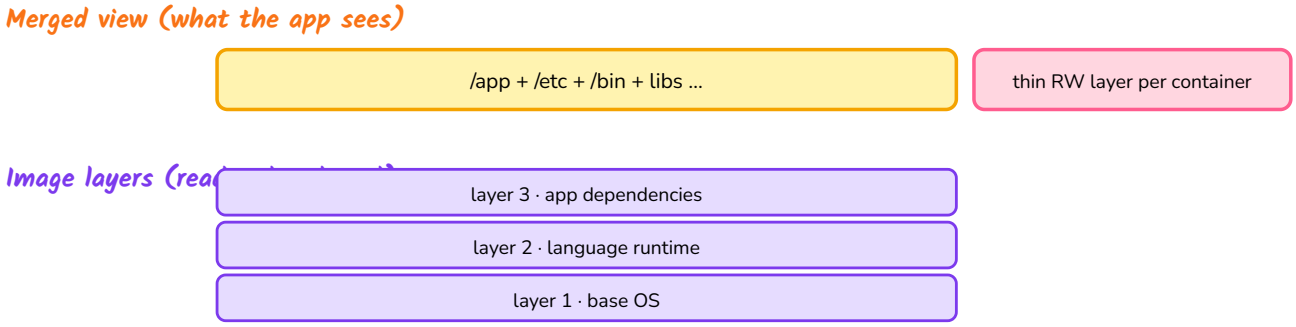


Namespaces map host resources so each container sees its own PID 1, filesystem, network, users...

🔍 Key namespace types

PID NET MNT UTS IPC USER — process IDs, networking, mounts, hostname, inter-process comms, and user mapping (e.g. map container root → a different host user).

Union filesystem — "how are layers shared?"



*Many containers from one image share the same base layers — only a thin **read/write overlay** is unique to each.*

💡 Why this matters

Efficient use of disk & network: pull a layer once, reuse everywhere. It's also why **layers are cached** at build time (Chapter 6) and why data written at runtime disappears when a container is removed (Chapter 4).

2.2 Docker application architecture

🖥️ Docker Desktop

Installed on **development systems**. Free for personal use; paid for some commercial cases. Provides:

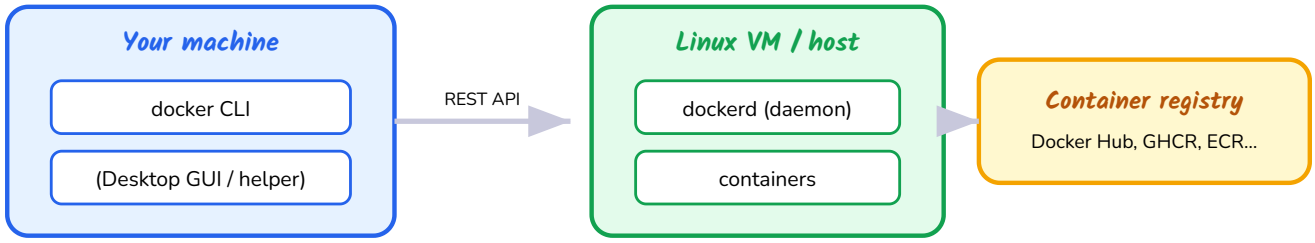
- Client: **Docker CLI**, GUI settings, credential helpers, extensions
- A **Linux VM** containing the Docker daemon (`dockerd`) + optional Kubernetes

🐧 Docker Engine

The free, open-source core — **Linux only**:

- **Docker CLI** (the `docker` command)
- **dockerd** — the daemon exposing the Docker API

Can build images and run containers, without the Desktop polish.



*CLI talks to **dockerd** over the Docker API; images are pulled from / pushed to a **registry**.*

🧠 Remember

Registries are not part of Docker. Docker operates Docker Hub, but GHCR, ECR, GCR, ACR, Harbor, Nexus and JFrog all speak the same standard — that's the whole point of a standard image format (Chapter 7).

🌟 Key Takeaways — Chapter 2

- **cgroups** = resource limiting/monitoring; **namespaces** = isolation; **union filesystem** = shared layers.
- Containers share the host kernel — these three features are why they're light and fast.
- **Docker Desktop** = CLI + GUI + Linux VM (dev convenience). **Docker Engine** = CLI + dockerd, Linux only.
- The CLI talks to **dockerd** via the Docker API; **registries** store/share images and are separate from Docker.
- Runtime writes go to a thin read/write layer unique to each container.

✅ Quick-Review Checklist

- ✅ Define cgroup, namespace, union FS
- ✅ Name 3 resources cgroups can limit
- ✅ Name 4 namespace types
- ✅ Describe overlay/shared-layer benefit
- ✅ Desktop vs Engine components
- ✅ What does dockerd expose?
- ✅ Are registries part of Docker?

📝 Practice Questions

Q1. **Which kernel feature limits a container's CPU and memory?**
→ **cgroups** (control groups).

Q2. **Which feature makes the container's PID 1 appear as PID 1 internally?**
→ **Namespaces** (PID namespace).

Q3. **Why are container image layers shared efficiently?**
→ Because of the **union filesystem**: identical layers are stored once and overlaid.

Q4. **Name the two components of Docker Engine.**
→ The **Docker CLI** and the **dockerd** daemon.

Q5. **What extra does Docker Desktop give you over Engine?**
→ A GUI, credential helpers, extensions, and a bundled Linux VM (plus optional Kubernetes).

Q6. What happens to files written inside a container when it is removed?
→ They're lost — they lived in that container's thin read/write layer. Use volumes to persist (Ch. 4).

Q7. Is Docker Hub the only container registry?
→ No — GHCR, ECR, GCR, ACR, Harbor, Nexus, JFrog and others exist; registries aren't part of Docker itself.

Docker · Part 3

3 Installation & Setup

Getting Docker running on your machine and launching your very first containers.

📖 What to install?

On a **development system** install **Docker Desktop**. On a **Linux server** install **Docker Engine**. (See Chapter 2 for the difference.)

🐳 Docker Desktop

Download & run the installer for your OS.
Docs: `docs.docker.com/get-docker`

🐧 Docker Engine (Linux)

One-liner install script:
`get.docker.com`

3.1 Configure Docker Desktop

Default settings are fine to start. As you run heavier stacks, tune the resources allocated to the Docker Linux VM (CPUs, memory, disk) inside the **Settings** panel of the GUI.

Tip:
If a big build feels slow or gets killed, it's often the VM's memory/CPU limit — not your machine. Bump it in Settings.

3.2 Running your first containers

🐳 Hello, World (whalesay)

```
# Run the classic Docker whale saying hello
docker run docker/whalesay cowsay "Hey Team! 👋"
```

docker run <image> <command> — downloads the image, starts a container, runs the command, prints output, exits.

🐘 Run Postgres

```
docker run \
  --env POSTGRES_PASSWORD=foobarbaz \
  --publish 5432:5432 \
  postgres:15.1-alpine
```

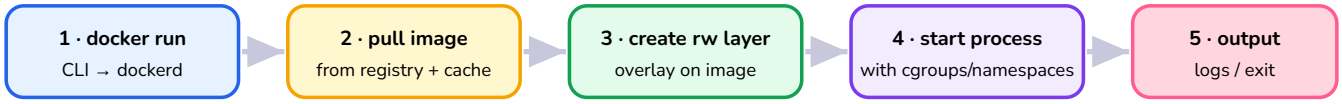
Publishes container port 5432 to host 5432, and sets the admin password via an env var.

🧩 Anatomy of `docker run`

- `docker run` — create + start a container
- `--env / -e` — set an environment variable
- `--publish / -p host:container` — map ports
- `--interactive -i` — keep STDIN open
- `--tty -t` — allocate a terminal
- `--rm` — delete container on exit
- `--name` — give it a friendly name
- last token — the image tag (e.g. `postgres:15.1-alpine`)

🚨 Watch out for

- Port already in use → change host port (`-p 5433:5432`).
- Image tag missing → Docker assumes `:latest` (avoid in prod!).
- Binaries/commands not found → install them **into the image**, not the running container.



What happens when you type `docker run`.

🌟 Key Takeaways — Chapter 3

- Dev machine → **Docker Desktop**; Linux server → **Docker Engine**.
- Tune CPU/memory/disk for the Docker VM in Desktop **Settings**.
- `docker run IMAGE COMMAND` pulls (if needed), creates, and starts a container.
- Common flags: `-e`, `-p`, `-it`, `--rm`, `--name`.

✅ Quick-Review Checklist

- ✅ Know which Docker edition for which OS
- ✅ Can run the whalesay hello-world
- ✅ Can run Postgres with a published port
- ✅ Know what `-e`, `-p`, `-it`, `--rm` do
- ✅ Know where to raise VM resources

📝 Practice Questions

Q1. Which edition should you install on a Linux server?
→ Docker Engine.

Q2. Write a command to run Postgres 15.1 with password "secret" and port 5432 exposed.
→ `docker run -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres:15.1-alpine`

Q3. What does `--rm` do?
→ Automatically removes the container when it exits.

Q4. What does `-p 8080:80` mean?
→ Map host port 8080 → container port 80.

Q5. A build is being killed for no obvious reason. First thing to check?
→ The Docker VM's allocated memory/CPU in Docker Desktop settings.

Q6. Why avoid relying on `:latest`?
→ The image can change underneath you → non-reproducible builds and surprise breakages.

Docker · Part 4

4 Using 3rd-Party Containers

Data persistence with volumes & bind mounts, then databases, throwaway OS/runtime sandboxes, and CLI tools.

You rarely build everything yourself. The fastest way to run Postgres, Mongo, Redis, a scratch Ubuntu box, or a specific version of `jq` is to pull someone else's image.

4.1 Understanding data persistence

🗂️ The core rule

Everything in an image is **read-only**. A fresh **read/write layer** is overlaid on top for each container. That layer lives and dies with the container.

A. Installing dependencies — and why runtime changes vanish

🧪 Experiment: install `ping` at runtime

```
docker run -it --rm ubuntu:22.04
ping google.com -c 1     # bash: ping: command not found
apt update && apt install iputils-ping --yes
ping google.com -c 1     # works!
exit

docker run -it --rm ubuntu:22.04
ping google.com -c 1     # fails again 🤔 — separate container, separate layer
```

🔁 Reusing a container

Give it a name and skip `--rm` so you can restart it and keep the change:

```
docker run -it --name my-ubuntu-container ubuntu:22.04
apt update && apt install iputils-ping --yes
exit
docker start my-ubuntu-container && docker attach my-ubuntu-container
ping google.com -c 1     # succeeds 🎉
```

🚨 The correct approach

Anything needed at runtime belongs **in the image**, not installed by hand:

```
docker build -t my-ubuntu-image —«EOF
FROM ubuntu:22.04
RUN apt update && apt install iputils-ping --yes
EOF
docker run -it --rm my-ubuntu-image
ping google.com -c 1     # pre-installed 😻
```

The one exception is **environment-specific config** (env vars, config files) — supply that at runtime.

B. Persisting application data — volumes & mounts

📦 Volume mount

Storage **managed by Docker** (on Linux: `/var/lib/docker/volumes`). Portable, fast, preferred.

📁 Bind mount

A directory from **your host filesystem** mapped inside. Great for visibility/config.

⚡ tmpfs mount

Lives in **memory**; gone when the container exits. For secrets/temp data you don't want persisted.

💾 Named volume — persist across containers

```
docker volume create my-volume
docker run -it --rm --mount source=my-volume,destination=/my-data ubuntu:22.04
# same thing, shorter flag:
docker run -it --rm -v my-volume:/my-data ubuntu:22.04

echo "Hello from the container!" > /my-data/hello.txt
exit
# new container, same volume → file still there
docker run -it --rm -v my-volume:/my-data ubuntu:22.04
cat my-data/hello.txt   # it persisted! ✅
```

🔗 Bind mount — from the host

```
docker run -it --rm --mount type=bind,source="${PWD}"/my-data,destination=/my-data
ubuntu:22.04
# shorter:
docker run -it --rm -v ${PWD}/my-data:/my-data ubuntu:22.04
echo "Hello!" > /my-data/hello.txt
cat my-data/hello.txt   # visible on host too
```

Peek:

On Docker Desktop, "the host" is really a **Linux VM**. A privileged helper container (`justincormack/nsenter1`, pinned by digest) lets you browse `/var/lib/docker/volumes` inside it.

4.2 Use case A — Databases

Databases are fiddly to install. Containers make it trivial to run any version, isolated and throwaway. Three rules: **volume(s) for data**, **bind mount(s) for config**, **env vars for settings**.

🐘 Postgres

```
docker run -d --rm \
  -v pgdata:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=foobarbaz \
  -p 5432:5432 \
  postgres:15.1-alpine
```

🍃 Mongo

```
docker run -d --rm \
  -v mongodata:/data/db \
  -e MONGO_INITDB_ROOT_USERNAME=root \
  -e MONGO_INITDB_ROOT_PASSWORD=foobarbaz
\
  -p 27017:27017 mongo:6.0.4
```

🧰 Redis

```
docker run -d --rm \
  -v redisdata:/data \
  redis:7.0.8-alpine
```

🐬 MySQL

```
docker run -d --rm \
  -v mysqldata:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=foobarbaz \
  -p 3306:3306 mysql:8.0.32
```

🔎 Elasticsearch

```
docker run -d --rm \
  -v
elasticsearchdata:/usr/share/elasticsearc
h/data \
  -e ELASTIC_PASSWORD=foobarbaz \
  -e "discovery.type=single-node" \
  -p 9200:9200 -p 9300:9300
elasticsearch:8.6.0
```

🕸️ Neo4j

```
docker run -d --rm \
  -v neo4jdata:/data \
  -e NEO4J_AUTH=neo4j/foobarbaz \
  -p 7474:7474 -p 7687:7687 \
  neo4j:5.4.0-community
```

🚨 Always verify the data path

The volume must point at the **exact in-container path** where that database writes. Double-check per version — a wrong path silently loses your data.

4.3 Use case B — Interactive test environments

🐧 Operating systems

```
docker run -it --rm ubuntu:22.04
docker run -it --rm debian:bullseye-slim
docker run -it --rm alpine:3.17.1
docker run -it --rm busybox:1.36.0
```

⌨️ Programming runtimes

```
docker run -it --rm python:3.11.1
docker run -it --rm node:18.13.0
docker run -it --rm php:8.1
docker run -it --rm ruby:alpine3.17
```

4.4 Use case C — CLI utilities

Run a specific tool version without installing anything on your host — just pipe stdin through the container.

Tool	Command
jq	`docker run -i stedolan/jq <test.json '.key_1 + .key_2'`
yq	`docker run -i mikefarah/yq <test.yaml '.key_1 + .key_2'`
sed (GNU)	`docker run -i --rm busybox sed 's/file./file!/g' <test.txt`
base64	`echo "…" | docker run -i --rm busybox base64`
aws-cli	`docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli s3 ls`
gcloud	`docker run --rm -v ~/.config/gcloud:/root/.config/gcloud gcr.io/…/google-cloud-cli gsutil ls`

💡 Improving ergonomics

Wrap frequent tools in a shell function or alias so they feel native:

```
yq() { docker run --rm -i -v ${PWD}:/workdir mikefarah/yq "$@"; }
# or
alias yq='docker run --rm -i -v ${PWD}:/workdir mikefarah/yq'
```

📏 Watch the size

Some images are huge — the Google Cloud CLI image is ~**2.8 GB**. Prefer slim/alpine variants where possible.

🌟 Key Takeaways — Chapter 4

- Image = read-only; each container gets its own **thin read/write layer** that vanishes on removal.
- Install dependencies **into the image**, not at runtime; pass secrets/config as runtime env.
- **Volumes** persist data (Docker-managed), **bind mounts** share host files, **tmpfs** is memory-only.
- For databases: volume the data dir, bind-mount config, set env vars — and verify the data path.
- Containers are ideal for throwaway OS/runtime sandboxes and pinned-version CLI tools.

✅ Quick-Review Checklist

- ✅ Explain why runtime installs disappear
- ✅ Volume vs bind vs tmpfs
- ✅ `--mount` vs `-v` syntax
- ✅ Run Postgres with a data volume
- ✅ Run a throwaway ubuntu shell
- ✅ Pipe input into a containerized CLI tool

📝 Practice Questions

Q1. Why does installing `ping` in one container not affect the next?
→ Each container has its own read/write layer; the image stays read-only.

Q2. Which mount type is Docker-managed and portable?
→ A named **volume** mount.

Q3. Give the short syntax to mount `pgdata` at `/var/lib/postgresql/data`.
→ `-v pgdata:/var/lib/postgresql/data`

Q4. You want a config file from your host read by the DB. Which mount?
→ A **bind mount** of that file into the container's config path.

Q5. What's the risk if you mount the volume at the wrong path?
→ Data is written elsewhere (to the container layer) and is lost when the container is removed.

Q6. How do you run `jq` against a host file without installing it?
→ `docker run -i stedolan/jq <test.json '.key_1 + .key_2'`

Q7. When is `tmpfs` appropriate?
→ For data you intentionally don't want to persist (e.g. credential files held in memory).

Q8. Why pin an image by digest like `@sha256:…`?
→ To guarantee you get that exact image, reducing supply-chain risk.

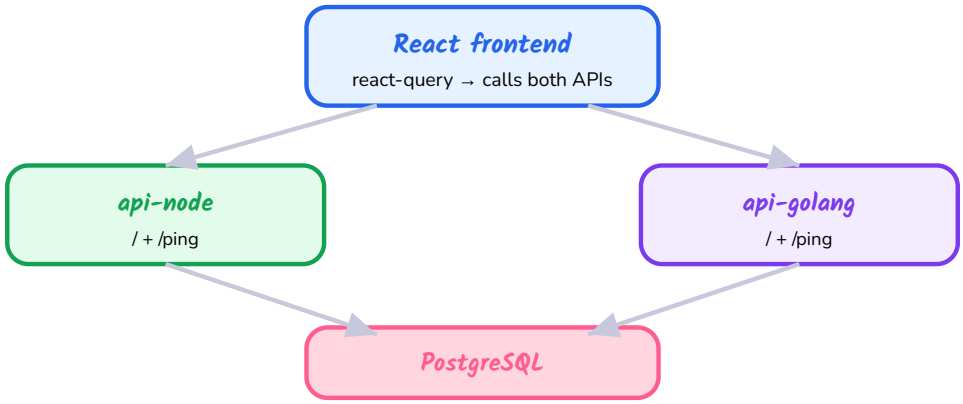
Docker · Part 5

5 Example Web Application

Meet the 3-tier app we'll containerize, compose, and deploy for the rest of the course.

We need a realistic app to work with — small enough to understand, real enough to hit every container concept.

5.1 The tech stack



Classic 3-tier: a browser client, two interchangeable API backends, and one database.

⚛ React frontend

Uses **react-query** to load data from the Node and Go APIs and render the results. Served in dev by **Vite**.

🔌 Node & Go APIs

Both expose `/` (queries the DB for the current time) and `/ping` (returns `pong`). Two languages on purpose — to contrast image builds later.

🐘 Postgres

An **empty** database — no tables. The APIs run `SELECT NOW() as now;` to prove connectivity.

5.2 Running it locally (the "boring way")

The whole course premise is that you often **don't** want to run & install the stack locally — that's what containers fix. But to have a baseline, here's how:

Tier	Prep	Run command
Postgres	—	`make run-postgres` (container, port 5432)
api-node	`npm install`	`make run-api-node` (nodemon dev mode)
api-golang	`go mod download`	`make run-api-golang` (build + run)
client-react	`npm install`	`make run-client-react` (Vite dev server)

🗒️ Why a Makefile?

One memorable command per tier keeps the local workflow consistent — and later mirrors what Compose will do declaratively.

💡 The pain we're about to remove

To run this the classic way you need Node, npm, Go, a database server, and matching versions of everything on every machine. **Containers package all of that** into reproducible images.

🌟 Key Takeaways — Chapter 5

- The demo app is a minimal **3-tier** app: React client → Node/Go APIs → Postgres.
- Two language backends let us compare Dockerfile strategies (Ch. 6).
- `/` hits the DB; `/ping` returns `pong` (health check later).
- Running it manually needs many host dependencies — the problem containers solve.

✅ Quick-Review Checklist

- ✅ Name the three tiers
- ✅ Which endpoints do the APIs expose?
- ✅ How is the DB queried?
- ✅ Why run Postgres in a container?
- ✅ What does each `make` target do?

📝 Practice Questions

Q1. What does the React client use to fetch data?
→ **react-query**, calling both API backends.

Q2. What does `/ping` return?
→ `pong` (useful as a liveness/health check later).

Q3. What SQL proves DB connectivity?
→ `SELECT NOW() as now;`

Q4. Why are there two API implementations?
→ To contrast building/shipping a Node image vs a compiled Go image.

Q5. What problem does containerizing this app solve?

→ It removes the need to install and version-match Node, Go, Postgres, etc. on every machine.

Docker · Part 6

6 Building Container Images

Dockerfiles from naive to production-grade: layers, caching, multi-stage builds, secrets, and more.

The image defines what your container *is*. Writing a great Dockerfile is the single highest-leverage skill in this whole course — it affects build speed 🏎️, security 🔒, and clarity 👁️.

6.1 The general process

A Dockerfile usually mirrors the steps you'd take to run an app on a fresh server:

1 · Start with an OS → **2 · Install the language runtime** → **3 · Install app dependencies** → **4 · Set up exec environment** → **5 · Run the app**

> 💡 **Shortcut**
>
> Often you can start at step 3 by choosing a base image that already bundles the OS + runtime (e.g. `node:19.6-bullseye-slim`, `golang:1.19-bullseye`, `python:3.11.1`).

6.2 Writing good Dockerfiles (the 10 techniques)

Legend: 🔒 security · 🏎️ build speed · 👁️ developer clarity

#	Technique	Why
1	**Pin a specific base image** (major+minor or SHA)	avoid surprise upstream changes 🔒👁️
2	**Choose a smaller base image** (slim/alpine)	smaller final image 🔒🏎️
3	**Choose a more secure base image** (e.g. Chainguard hardened)	fewer CVEs, smaller attack surface 🔒
4	**Specify a WORKDIR** following language convention	predictable layout 👁️
5	**Respect layer caching** — order steps by frequency of change	faster rebuilds 🏎️
6	**Use COPY --link** where appropriate	copy into an independent layer 🏎️
7	**Use a non-root USER**	defense-in-depth 🔒
8	**Specify the environment correctly** (prod deps, NODE_ENV...)	smaller + correct 🔒🏎️
9	**Avoid assumptions** — use EXPOSE, ENV, LABEL	self-documenting 👁️
10	**Use multi-stage builds** where sensible	drastically smaller images 🔒🏎️

🏎️ Protect the layer cache

- Order commands by **frequency of change** (rarely-changing first).
- Copy the **dependency manifest** → install deps → copy the rest of the source.
- Use **cache mounts** (`--mount=type=cache`).
- Use **COPY --link**.
- Combine always-linked steps; use **heredocs** for tidiness.

🔒 Keep it clean & safe

- Use **.dockerignore** and COPY only what's needed.
- Use a **non-root USER**.
- Install **only production** deps in the prod image.
- **Never bake secrets** into layers (they persist in history!).

6.3 Instruction reference

Instruction	Purpose
`FROM`	base image (first stage or a named stage `AS name`)
`RUN`	execute during **build**; creates a new layer
`COPY` / `ADD`	copy files in from build context (prefer `COPY` ; `ADD` also unpacks archives/fetches URLs)
`WORKDIR`	set the working directory for later instructions
`ENV` / `ARG`	env var persisting into the image / build-time-only variable
`CMD` / `ENTRYPOINT`	**run-time** defaults (see §6.4)
`EXPOSE`	document the intended port (metadata only)
`USER`	switch to a non-root user for the rest
`LABEL`	attach metadata (authors, OCI labels, source, version)
`VOLUME`	declare a mount point
`HEALTHCHECK`	tell Docker how to test container health

🐚 RUN shell form vs exec form

```
RUN echo "Hey Team 👋"          # shell form → /bin/sh -c "..."
RUN ["echo", "Hey Team 👋"]      # exec form  → runs directly, no shell
```

6.4 ENTRYPOINT vs CMD — the classic gotcha

⚙️ Given an image with...

```
ENTRYPOINT [ "echo", "Hey Team 👋 (entrypoint)" ]
CMD [ "+ (cmd)" ]
```

You run...	Effective command
`docker run sample`	ENTRYPOINT + CMD
`docker run sample "+ (argument)"`	ENTRYPOINT + argument (**CMD ignored**)
`docker run --entrypoint echo sample`	new entrypoint + CMD
`docker run --entrypoint echo sample "hi"`	new entrypoint + argument

🧠 Remember

CMD = default arguments that are **easily overridden**. **ENTRYPOINT** = the fixed command. Pass arguments at run time and they **replace CMD**, not ENTRYPOINT.

6.5 Extra Dockerfile features

📜 Parser directives

Must be the **very first** thing in the file:

```
# syntax=docker/dockerfile:1.5
# escape=\
```

🧵 Heredocs

Multi-line single step without `&&` / `\`:

```
RUN <<EOF
apt update
apt install iputils-ping -y
EOF
```

🔑 Build secrets

Keep credentials out of the final image:

```
RUN --
mount=type=secret,id=secret.txt,dst=/cont
ainer-secret.txt \
  echo "use the secret here"
```

🧱 buildx (multi-arch)

One Dockerfile → many platforms:

```
docker buildx build \
  --platform
linux/amd64,linux/arm64,linux/arm/v7 \
  -t repo/img:latest --push .
```

🏗️ Build & run the sample

```
# build with a build-arg and a secret
DOCKER_BUILDKIT=1 docker build \
  --build-arg BASE_IMAGE_TAG=19.3 \
  --secret id=secret.txt,src=local-secret.txt \
  -t sample -f Dockerfile.sample .

# see ENTRYPOINT/CMD interaction live
make run-sample-entrypoint-cmd
```

6.6 Learning by evolution: Dockerfile.0 → Dockerfile.N

Each service folder contains a ladder of Dockerfiles, from naive to polished. Final ones are what we deploy.

🐣 api-node · Dockerfile.0 — the naive start

```
FROM node
COPY . .
RUN npm install
CMD [ "node", "index.js" ]
```

Problems: unpinned + huge base, copies everything (breaks cache), runs as root, dev deps included.

✅ api-node · Dockerfile.4 — much better

```
FROM node:19.6-bullseye-slim        # pinned + slim
WORKDIR /usr/src/app
COPY package*.json ./               # cache-friendly
RUN npm install
USER node                           # non-root
COPY --chown=node:node ./src/ .     # only needed files
CMD [ "node", "index.js" ]
```

🚀 api-node · Dockerfile.9 — multi-stage (dev + production)

```
FROM node:19.6-bullseye-slim AS base
WORKDIR /usr/src/app
COPY package*.json ./

FROM base AS dev
RUN --mount=type=cache,target=/usr/src/app/.npm \
  npm set cache /usr/src/app/.npm && npm install
COPY . .
CMD ["npm", "run", "dev"]

FROM base AS production
ENV NODE_ENV production
RUN --mount=type=cache,target=/usr/src/app/.npm \
  npm set cache /usr/src/app/.npm && npm ci --only=production
USER node
COPY --chown=node:node ./healthcheck/ .
COPY --chown=node:node ./src/ .
EXPOSE 3000
CMD [ "node", "index.js" ]
```

Shared `base` *stage;* `dev` *for hot-reload,* `production` *with prod-only deps + non-root.*

🐹 api-golang · Dockerfile.8 — build → `scratch`

```
FROM golang:1.19-bullseye AS build-base
WORKDIR /app
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/go/pkg/mod \
  --mount=type=cache,target=/root/.cache/go-build \
  go mod download

FROM build-base AS build-production
RUN useradd -u 1001 nonroot
COPY . .
RUN go build -ldflags="-linkmode external -extldflags -static" \
  -tags netgo -o api-golang
RUN go build -ldflags="-linkmode external -extldflags -static" \
  -tags netgo -o healthcheck ./healthcheck/healthcheck.go

FROM scratch                              # tiny final image
ENV GIN_MODE=release
WORKDIR /
COPY --from=build-production /etc/passwd /etc/passwd
COPY --from=build-production /app/healthcheck healthcheck
COPY --from=build-production /app/api-golang api-golang
USER nonroot
EXPOSE 8080
CMD ["/api-golang"]
```

Compile in a builder stage, ship only the static binary on **scratch** *— a few MB.*

client-react · Dockerfile.5 — build then serve with nginx

```
FROM node:19.4-bullseye AS build
WORKDIR /usr/src/app
COPY package*.json ./
RUN --mount=type=cache,target=/usr/src/app/.npm \
  npm set cache /usr/src/app/.npm && npm install
COPY . .
RUN npm run build

FROM nginxinc/nginx-unprivileged:1.23-alpine-perl
COPY --link nginx.conf /etc/nginx/conf.d/default.conf
COPY --link --from=build usr/src/app/dist/ /usr/share/nginx/html
EXPOSE 8080
```

Build the static bundle, then serve it from a tiny unprivileged nginx. Node never ships to prod.

Build:

`cd api-golang && N=4 make build-N` builds `Dockerfile.4`. Every version still runs; the highest number is the deployed one.

🌟 Key Takeaways — Chapter 6

- Dockerfile steps mirror server setup: OS → runtime → deps → env → run.
- Pin versions, pick small/secure bases, set WORKDIR, use non-root, be explicit.
- **Order matters for caching:** copy manifests → install → copy source.
- **CMD** is overridable defaults; **ENTRYPOINT** is the fixed command; runtime args replace CMD.
- **Multi-stage builds** isolate build tooling and shrink the shipped image (down to `scratch`).
- Use build secrets + `.dockerignore`; never bake secrets into layers.
- Parser directives, heredocs, ARG, and `buildx` multi-arch are essential tools in your kit.

✅ Quick-Review Checklist

- [x] List the 5 general build steps
- [x] Explain layer-cache-friendly ordering
- [x] CMD vs ENTRYPOINT differences
- [x] Shell vs exec RUN form
- [x] ARG vs ENV
- [x] What a multi-stage build is
- [x] How to pass build secrets safely
- [x] What `.dockerignore` is for
- [x] buildx multi-arch command

📝 Practice Questions

Q1. Why copy `package*.json` before the source code?

→ So the dependency-install layer is cached and only re-runs when deps change, not on every source edit.

Q2. Difference between `ARG` and `ENV`?
→ `ARG` is build-time only (still visible in metadata); `ENV` persists into the running container.

Q3. With `ENTRYPOINT ["echo","A"]` and `CMD ["B"]`, what does `docker run img C` execute?
→ `echo A C` — CMD is replaced by the runtime argument.

Q4. What's the point of a multi-stage build?
→ Keep build tooling out of the final image → smaller, more secure, faster-to-pull images.

Q5. Why is `--mount=type=secret` better than `ARG` / `ENV` for secrets?
→ The secret isn't written into any image layer or metadata — it exists only during the build step.

Q6. Why use `scratch` as the final base for Go?
→ A statically linked binary needs no OS files; the resulting image is tiny and has a minimal attack surface.

Q7. What does `COPY --link` improve?
→ It copies into an independent layer so cache isn't invalidated by changes in earlier/later steps as easily.

Q8. Why pin a base image by SHA256 digest?
→ Tag can move; a digest guarantees the exact immutable image for reproducibility and supply-chain safety.

Docker · Part 7

7 Container Registries

Where images live: storing, sharing, and authenticating to registries across the ecosystem.

📚 What is a registry?

A **repository (or collection of repositories)** used to **store and access container images**. They're the meeting point between developer machines, CI servers, and deployment environments.

7.1 The popular registries

🐳 General purpose

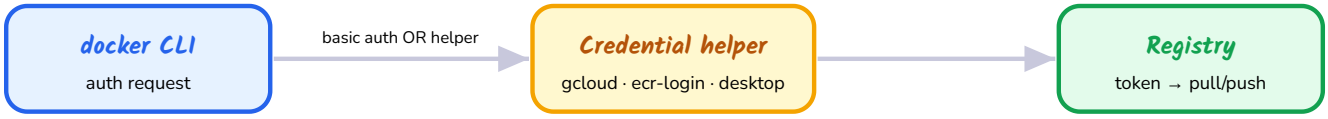
Docker Hub · GitHub GHCR · GitLab Registry

☁️ Cloud / enterprise

Google GCR · AWS ECR · Azure ACR · JFrog · Nexus · Harbor

7.2 Authentication

Pulling public images needs no login, but to **push** an image or pull a **private** one, you must authenticate.



*Docker can log in with username/password, or delegate to a **credential helper** (e.g. `gcloud` for GCR).*

🔑 Login & push

```
docker login
docker tag myapp:1.0 registry/myapp:1.0
docker push registry/myapp:1.0
docker pull registry/myapp:1.0
```

🔐 Credential storage

Credentials can be kept in a **secure store** such as the macOS Keychain or Windows Credential Manager — better than plaintext `~/.docker/config.json`.

🚨 *Image naming tells you the registry*

`nginx` = Docker Hub official. `ghcr.io/org/app` = GitHub. `<id>.dkr.ecr.<region>.amazonaws.com/app` = AWS ECR. The registry is the first part of the fully-qualified image name.

🌟 Key Takeaways — Chapter 7

- A registry stores & shares images between dev, CI, and deployment.
- Public pulls need no auth; **push** and **private pulls** require login.
- Docker uses basic auth **or** credential helpers (e.g. `gcloud`, `ecr-login`).
- Credentials can live in the OS secure store.
- Registries are a standard, not a Docker-only thing — Hub, GHCR, GCR, ECR, ACR, Harbor...

✅ Quick-Review Checklist

- ✅ Define a container registry
- ✅ Name 4 registries
- ✅ When is auth required?
- ✅ What is a credential helper?
- ✅ login / tag / push / pull flow

📝 Practice Questions

Q1. **When must you authenticate to a registry?**
→ To push any image, or to pull a private image.

Q2. **What is a credential helper?**
→ An external program Docker calls to obtain registry credentials (e.g. `gcloud` for GCR).

Q3. **How does Docker know which registry an image belongs to?**
→ From the first part of the image name (hostname), e.g. `ghcr.io/…`.

Q4. **Where can credentials be securely stored?**
→ macOS Keychain / Windows Credential Manager (via Docker's credential-store config).

Q5. **Give the command sequence to publish `myapp:1.0` to Docker Hub.**
→ `docker login` → `docker tag myapp:1.0 you/myapp:1.0` → `docker push you/myapp:1.0`

Docker · Part 8

8 Running Containers

`docker run` vs `docker compose up` — and the configuration flags you must know.

Two primary ways to run containers: a one-shot `docker run`, or a declarative `docker compose up` for multi-service apps.

🏃 docker run

Takes **one image** and runs one container. Perfect for quick, one-off use cases (like the tools in Chapter 4).

```
docker run -d --name db \
  -e POSTGRES_PASSWORD=foobarbaz \
  -v pgdata:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:15.1-alpine
```

🎼 docker compose

Takes a spec of **1+ services**, can **build** their images and **run** their containers. Best for multi-service apps.

```
docker compose up --build
docker compose down
```

docker run
imperative · one container · great for ad-hoc

docker compose
declarative · many services · in one file

*Rule of thumb: one-off → **run**; app with more than one service → **compose**.*

8.1 Why `docker run` gets messy for apps

Running all four services by hand needs a long list of carefully-set options:

- Uses a shared **bridge network** so containers can find each other.
- Uses `--link` (legacy) to get easy hostnames for connections.
- **Publishes ports** (`-p`) to reach services from the host — really only the frontend needs it.
- **Named containers** are easier to reference, but you must remove them to avoid name clashes.
- **Restart policy** lets Docker restart a container (e.g. an API that crashed because the DB wasn't ready yet).

😖 The pain

Correct ordering, teardown, network creation, and env vars all become manual shell chores. Compose encodes it all in one file.

8.2 The whole stack in one compose file

docker-compose.yml (abridged)

```
services:
  client-react-nginx:
    image: client-react-nginx
    build:
      context: ../05-example-web-application/client-react/
      dockerfile: ../../06-building-container-images/client-react/Dockerfile.5
    init: true
    networks: [frontend]
    ports: [80:8080]
    restart: unless-stopped

  api-node:
    image: api-node
    build: { context: ../05-example-web-application/api-node/,
             dockerfile: ../../06-building-container-images/api-node/Dockerfile.7 }
    init: true
    depends_on: [db]
    environment:
      - DATABASE_URL=postgres://postgres:foobarbaz@db:5432/postgres
    networks: [frontend, backend]
    ports: [3000:3000]
    restart: unless-stopped

  api-golang:
    image: api-golang
    build: { context: ../05-example-web-application/api-golang/,
             dockerfile: ../../06-building-container-images/api-golang/Dockerfile.6 }
    depends_on: [db]
    environment:
      - DATABASE_URL=postgres://postgres:foobarbaz@db:5432/postgres
    networks: [frontend, backend]
    ports: [8080:8080]
    restart: unless-stopped

  db:
    image: postgres:15.1-alpine
    volumes: [pgdata:/var/lib/postgresql/data]
    environment: [POSTGRES_PASSWORD=foobarbaz]
    networks: [backend]
    ports: [5432:5432]

volumes:
  pgdata:
networks:
  frontend:
  backend:
```

Two networks model the tiers: the client & APIs share `frontend`; only the APIs & DB share `backend`.

💡 What compose gives you for free

- Builds + runs with a single command.
- Automatic container/image naming.
- Aggregated logs from all services.
- `depends_on` ordering, shared networks, and volumes declared once.

8.3 Flags everyone should know

Flag	Meaning
`-d`	detached (run in background)
`--entrypoint`	override the image ENTRYPOINT
`-e / --env / --env-file`	set environment variables
`--init`	run an init process as PID 1 (handle signals/reaping)
`-i / -t`	interactive STDIN / allocate a TTY
`--mount / -v`	mount a volume or bind
`--name`	name the container
`--network / --net`	attach to a network
`--platform`	force a target platform (e.g. linux/amd64)
`-p / --publish`	publish ports host:container
`--restart`	restart policy (e.g. `unless-stopped`)
`--rm`	remove container on exit

🛠️ Less common, still worth knowing

`--cap-add / --cap-drop` `--cpuset-cpus (pin cores)` `--cpu-shares` `--memory / -m` `--gpus (NVIDIA)` `--health-cmd ...` `--pid / --pids-limit` `--privileged` `--read-only` `--security-opt` `--userns` `--device-*-bps / -iops`

🌟 Key Takeaways — Chapter 8

- `docker run` = one image/container (ad-hoc); `compose` = multi-service app as code.
- Compose encodes builds, networks, volumes, env, ordering, and restart policy in one file.
- You can express **any** `docker run` flag in a compose file.
- Names matter for DNS: services reach each other by service name (e.g. host `db`).
- Every flag is documented at `docs.docker.com/engine/reference/run/`.

✅ Quick-Review Checklist

- When to prefer run vs compose
- Read a compose file: services, networks, volumes
- Know `-d, -e, -p, -v, --name, --network, --restart`
- What `depends_on` does (and doesn't)
- Publish ports only where needed

📝 Practice Questions

Q1. Best tool for running an app with 5 services?
→ `docker compose` — declarative, one file, aggregated logs.

Q2. What does `restart: unless-stopped` protect against?
→ Containers crashing (e.g. API starts before the DB is ready) — Docker restarts them.

Q3. Why publish only the frontend's port?
→ Only the entrypoint needs host access; internal services reachable via the shared network.

Q4. What does `--init` do?
→ Runs a tiny init as PID 1 to reap zombies and forward signals properly.

Q5. How does the Node API reach the DB in compose?
→ By service name as hostname: `postgres://…@db:5432/postgres`.

Q6. Can you set CPU/memory limits via compose?
→ Yes — any run flag has a compose equivalent (e.g. `deploy.resources` / service limits).

Docker · Part 9

9 Container Security

Two fronts: the contents of your image, and the security of how you run it.

Security splits into two questions: **(1)** what vulnerabilities exist in my image? and **(2)** if an attacker gets in, what can they do?

📦 Image security

"What vulnerabilities exist in your image that an attacker could exploit?"

🏃 Runtime security

"If a container is compromised, what can the attacker do, and how easily can they move laterally?"

9.1 Image security

- **Minimise attack surface** — use minimal base images (multi-stage builds help a lot) and don't install what you don't need (no dev deps).
- **Scan images!** for known CVEs.
- Use users with **minimal permissions**.
- Keep **sensitive info out** of images entirely.
- **Sign and verify** images.
- **Fix tags** — pin `major.minor` (still allows patches) or a specific digest.

9.2 Runtime security

⚙️ Docker daemon (dockerd)

Start with `--userns-remap` so root inside a container maps to a non-root user on the host (`docs.docker.com/engine/security/userns-remap/`).

🛡️ Harden individual containers

- Use a **read-only filesystem** (`--read-only`) when writes aren't needed.
- `--cap-drop=all` , then `--cap-add` only what's required.
- Limit CPU & memory: `--cpus="0.5" --memory 1024m` .
- Apply `--security-opt` via **seccomp** and **apparmor** profiles.

💻 Hardened run command

```
docker run -d \
  --read-only \
  --cap-drop=all --cap-
add=NET_BIND_SERVICE \
  --cpus="0.5" --memory 1024m \
  --security-opt no-new-privileges \
  --user 1001 \
  myapp:1.0
```

🚨 The golden rules

Don't bake secrets into images · don't run as root · don't give more capabilities than needed · don't trust tags that can move · don't ship build tools to prod.

🌟 Key Takeaways — Chapter 9

- Security = **image contents** + **runtime configuration**.
- Shrink attack surface: minimal bases, no dev deps, multi-stage.
- Scan, sign, and verify images; keep secrets out; pin tags/digests.
- `--userns-remap` on the daemon; per-container hardening with read-only FS, cap-drop, limits, seccomp/apparmor.

✅ Quick-Review Checklist

- ✅ The two security fronts
- ✅ How to reduce image attack surface
- ✅ Why scan & sign images
- ✅ What `--userns-remap` does
- ✅ Container hardening flags
- ✅ seccomp / apparmor purpose

📝 Practice Questions

Q1. **Name the two areas of container security.**
→ Image security and runtime security.

Q2. **Which build technique most reduces image attack surface?**
→ Multi-stage builds (ship only what's needed; exclude build tools/dev deps).

Q3. **What does `--cap-drop=all --cap-add=…` achieve?**
→ Removes all Linux capabilities, then adds back only the specific ones the container needs.

Q4. **Why might you run a container with `--read-only`?**
→ If it doesn't need to write to its filesystem, read-only prevents tampering with files.

Q5. **Why is `--userns-remap` valuable?**
→ It maps container root to an unprivileged host user, containing a breakout.

Q6. **What do seccomp and apparmor control?**
→ seccomp restricts syscalls; apparmor restricts program capabilities/profiles.

Q7. **Why pin `major.minor` rather than `latest`?**
→ You get security patches but avoid surprise breaking changes; a digest pins exactly.

Docker · Part 10

10 Interacting with Docker Objects

A working map of the Docker CLI: images, containers, volumes, and networks.

Know the CLI well and you'll debug faster than anyone. Two habits: read `docs.docker.com/engine/reference/commandline/cli/` and lean on `--help` (e.g. `docker build --help`).

> 💡 **Grouping**
>
> Commands are grouped by object type: `docker image …`, `docker container …`, `docker volume …`, `docker network …`. Short forms exist (`docker build` = `docker image build`, `docker run` = `docker container run`).

10.1 Images — `docker image COMMAND`

Command	What it does
`build`	Build an image from a Dockerfile
`history`	Show the layer history of an image
`import`	Create a filesystem image from a tarball
`inspect`	Detailed info on one or more images
`load`	Load an image from a tar archive or STDIN
`ls`	List images
`prune`	Remove unused images
`pull` / `push`	Pull from / push to a registry
`rm`	Remove one or more images
`save`	Save images to a tar archive
`tag`	Create a new tag referring to a source image

🔎 Scanning

```
docker scan IMAGE
# or a 3rd-party scanner, e.g. Trivy:
trivy image IMAGE
```

✍️ Signing (supply-chain)

```
docker trust sign IMAGE:TAG
docker trust inspect --pretty IMAGE:TAG
```

10.2 Containers — `docker container COMMAND`

📋 Lifecycle & info

- `run` / `create` / `start`
- `stop` / `kill` / `restart`
- `pause` / `unpause` / `wait`
- `rm` / `prune` / `rename`
- `ls` / `inspect` / `stats`
- `top` / `port` / `update`

🛠️ Interacting & files

- `attach` — attach local STDIO
- `exec` — run a command in a running container
- `logs` — fetch logs
- `cp` — copy files in/out
- `diff` — changes to the filesystem
- `commit` — image from a container's changes
- `export` — container FS as a tarball

💻 Everyday container commands

```
docker container ls -a                # all containers (incl. stopped)
docker container logs -f api-node     # follow logs
docker container exec -it db psql -U postgres   # shell into a running container
docker container stats                # live CPU/mem/IO
docker container cp db:/tmp/dump.sql . # copy a file out
docker container inspect api-node     # JSON details
docker container prune                # remove stopped containers
```

10.3 Volumes & Networks

📦 `docker volume COMMAND`

- `create` — create a volume
- `ls` — list volumes
- `inspect` — details
- `rm` — remove one/more
- `prune` — remove all unused

🌐 `docker network COMMAND`

- `create` — create a network
- `ls` — list networks
- `inspect` — details
- `connect` / `disconnect` — attach/detach a container
- `rm` / `prune`

🌉 Bridge networks & DNS

User-defined bridge networks provide **automatic service discovery by name** — that's how compose services reach each other by service name (e.g. host `db`). Legacy `--link` did this manually.

🌟 Key Takeaways — Chapter 10

- Docker CLI is grouped by object: image, container, volume, network.
- Learn the lifecycle + inspection commands by heart.
- `docker scan` / Trivy find CVEs; `docker trust` signs/verifies images.
- User-defined bridge networks give name-based DNS between containers.

- `--help` is always available when you forget a flag.

✅ Quick-Review Checklist

- [x] List images / containers / volumes / networks
- [x] Inspect any object
- [x] Follow logs & exec into a container
- [x] Scan and sign an image
- [x] Prune unused objects
- [x] Create/connect a network

📝 Practice Questions

Q1. Command to open an interactive shell in a running container?
→ `docker container exec -it NAME sh`

Q2. How do you follow a container's logs live?
→ `docker container logs -f NAME`

Q3. Which command shows an image's layers?
→ `docker image history IMAGE`

Q4. How do you scan an image for vulnerabilities?
→ `docker scan IMAGE` or a third-party scanner like Trivy.

Q5. What creates an image from a container's changes?
→ `docker container commit` .

Q6. How do containers resolve each other by name?
→ Via a user-defined bridge network's built-in DNS.

11 Development Workflows

Fast inner loops: hot reload, debuggers, tests — then a CI image pipeline.

Containers must not slow you down while coding. The goal: **iterate without rebuilding** the image, and still get auto-reload, debugging, and tests.

11.1 Five attributes of a great dev environment

1️⃣ Easy to set up

Whole environment described by one compose file. Teammates run a single command: `make compose-up-build` (or `…-debug`).

2️⃣ Iterate without rebuilding

Bind-mount your host source into the container so edits appear instantly:

```
- type: bind
  source: ../05-example-web-application/api-node/
  target: /usr/src/app/
```

3️⃣ Automatic reloading

React: Vite handles it. **Node:** `nodemon`. **Go:** `air` (watches & rebuilds).

4️⃣ A real debugger

React: browser devtools + react-query devtools. **Node:** `--inspect` → ws :9229. **Go:** `delve`. See §11.2.

5️⃣ Run the tests

Overlay a `docker-compose-test.yml` that swaps the command to run test suites; `make run-tests` combines it with the dev file.

🧩 Compose overlays

Pass multiple files to `docker compose up -f a.yml -f b.yml`; Docker **merges** them, later files overlaying earlier ones. Keeps dev/debug/test concerns separate.

11.2 Debugging setup per language

Service	Tool	Key details
React client	Browser devtools	+ `react-query-devtools` for query state
Node API	`--inspect`	listen on `0.0.0.0`, publish `9229`
Go API	`delve`	installed in `Dockerfile.dev`; command overridden in the debug compose

🧷 VSCode `launch.json` (the key bit)

```
{
  "name": "Attach to Node in Docker",
  "type": "node",
  "request": "attach",
  "port": 9229,
  "remoteRoot": "/usr/src/app",
  "localRoot": "${workspaceFolder}",
  "substitutePath": [
    { "from": "${workspaceFolder}", "to": "/usr/src/app" }
  ]
}
```

`substitutePath` *lets breakpoints in* ***local*** *files bind inside the container.*

11.3 Continuous Integration

See `.github/workflows/image-ci.yml` — a basic GitHub Action that **builds, scans, tags, and pushes** an image.

⚙️ The marketplace actions it uses

- `docker/metadata-action` — generate tags
- `docker/login-action` — log into Docker Hub
- `docker/build-push-action` — build & push
- `aquasecurity/trivy-action` — scan for vulnerabilities

🏷️ Typical tag strategy

```
type=raw,value=latest
type=ref,event=branch
type=ref,event=pr
type=semver,pattern={{version}}
type=semver,pattern={{major}}.{{minor}}
type=raw,value={{date 'YYYYMMDD'}}-{{sha}}
```

🔒 Fail CI on critical vulns

```
- uses: aquasecurity/trivy-action@master
  with:
    image-ref: 'repo/app:latest'
    format: 'table'
    exit-code: '1'
    ignore-unfixed: true
    severity: 'CRITICAL'
```

🌟 Key Takeaways — Chapter 11

- Bind-mount source + watcher tools = **iterate without rebuilding images**.
- Auto-reload: Vite (React), nodemon (Node), air (Go).
- Remote debuggers via published ports + `substitutePath`.
- Compose **overlays** keep dev / debug / test configs modular.
- CI pipeline: metadata → login → build&push → Trivy scan (fail on CRITICAL).

✅ Quick-Review Checklist

- [x] 5 dev-env attributes
- [x] How bind mounts help
- [x] Hot-reload tool per language
- [x] How to attach a remote debugger
- [x] What compose overlays do
- [x] Steps in the image CI workflow

📝 Practice Questions

Q1. **How do you change code without rebuilding the image?**
→ Bind-mount the host source directory into the container.

Q2. **Which tool hot-reloads the Go API?**
→ `air`.

Q3. **What port does the Node debugger use, and what must be configured?**
→ `9229`; listen on `0.0.0.0` and publish the port.

Q4. **What does `substitutePath` enable?**
→ Mapping local paths to container paths so breakpoints in your editor work inside the container.

Q5. **What is a compose overlay?**
→ A second compose file merged on top of the base (e.g. to add debug or test config).

Q6. **What are the four CI steps in `image-ci.yml`?**
→ Generate tags → log in → build & push → Trivy scan.

Docker · Part 12

12 Deploying Containers

Three deployment paths: Railway.app, Docker Swarm, and Kubernetes.

Containers standardised deployment, so there are **tons** of places to run them. Within AWS alone there were **17 ways to run containers** (2021 — likely more now). We deploy the same app three ways.

🚂 Railway.app

Painless PaaS. GitHub auto-deploy, nice UI, and **Nixpacks** to build images without a Dockerfile.

🐝 Docker Swarm

Compose's production sibling. Native **secrets** and **zero-downtime** redeploys. Single-node cluster in one command.

☸ Kubernetes

The most mature orchestrator with the broadest cloud support. More verbose, but the industry standard.

12.1 Railway.app

- Deploys from GitHub; **Nixpacks** can build an image with no Dockerfile (worked for the Go API).
- When you need control, provide your own Dockerfile.
- Constraint (Feb 2023): build context must equal the Dockerfile's location; no `--mount=type=cache`.
- Because of that, a `Dockerfile.railway` lives on the `railway` git branch.
- Config is declared in `railway.toml` (also on that branch).
- No private networking yet → an `nginx-railway.conf` uses the APIs' **public domains** instead of Docker DNS.
- Set `PORT` per service (80 nginx, 3000 node, 8080 golang). Railway auto-injects `DATABASE_URL` for Postgres.

12.2 Docker Swarm

🚧 Why not compose in production?

Two limitations: **no support for secrets** (we passed DB creds as env vars) and **no zero-downtime redeploys**.

✅ What Swarm adds

Native **secrets** (read as files inside apps) and rolling/zero-downtime updates — with only small compose edits (an added `deploy:` block). Initialise a single node with `docker swarm init`.

🚀 Deploy steps

```
1. Create a VM (open ports 80, 443, 22)
2. Install Docker Engine (get.docker.com)
3. Set DOCKER_HOST=USERNAME@IP in the Makefile ← local client → remote daemon
4. Build & push images to a registry
5. make create-secrets
6. make swarm-deploy-stack   # docker stack deploy under the hood
7. Add a DNS A record → your VM
```

Neat:

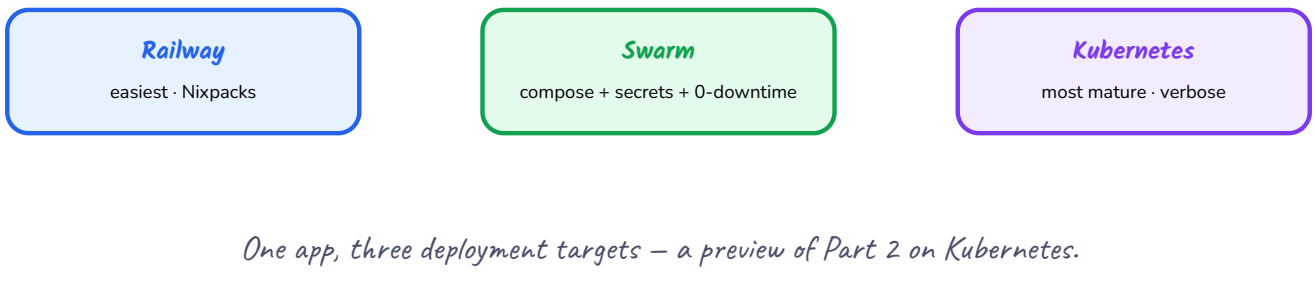
setting `DOCKER_HOST` lets your **local** Docker client drive a **remote** daemon — the deployment "just works" from your laptop.

12.3 Kubernetes (a taste)

Not complete without the most popular orchestrator. Kubernetes schedules & runs containers with more maturity and cloud-provider support than Swarm — but its manifests are **more verbose** than the swarm spec.

Deploy steps

```
1. Create a cluster (any cloud)
2. Configure kubectl to talk to it
3. make install-traefik    # ingress controller (Helm chart)
4. make install-postgres   # Bitnami Helm chart
5. make deploy-app
6. Point DNS at the Traefik load-balancer IP
```



One app, three deployment targets — a preview of Part 2 on Kubernetes.

🌟 Key Takeaways — Chapter 12

- Containers = a standard interface → a huge menu of deployment targets.
- **Railway** is fastest (Nixpacks, GitHub deploys) but has build-context/private-networking limits.

- **Swarm** = compose + secrets + zero-downtime; start with `docker swarm init`.
- Set `DOCKER_HOST` to deploy to a remote daemon from your machine.
- **Kubernetes** is the industry standard — more setup (Traefik, Postgres via Helm), more power.

✅ Quick-Review Checklist

- Three deployment options
- What Nixpacks does
- Why compose isn't ideal in prod
- What Swarm adds
- What `DOCKER_HOST` enables
- Kubernetes deploy flow

📝 Practice Questions

Q1. **What technology lets Railway build images without a Dockerfile?**
→ Nixpacks.

Q2. **Two limitations of plain compose for production?**
→ No secrets support and no zero-downtime redeploys.

Q3. **How do you initialise a single-node Swarm?**
→ `docker swarm init`.

Q4. **What does setting `DOCKER_HOST` do?**
→ Points your local Docker client at a remote Docker daemon for deployment.

Q5. **What do you install on Kubernetes before the app?**
→ Traefik (ingress) and Postgres (via Helm charts).

Q6. **Why offer Kubernetes at all?**
→ It's the most popular orchestrator with the strongest cloud-provider maturity.

Kubernetes · Part 1

1 History & Motivation

Three deployment eras — and why we needed an orchestrator.

To understand Kubernetes, follow the story: how we deployed software kept getting more abstract — and more complex. Each era solved the previous one's pain and created a new one.

2000s · Traditional Deployment Era

- **On-prem** data centers / colocation
- Teams of **sysadmins** provisioning fleets by hand
- Apps on **bare metal** servers
- **Monolithic** architecture
- Homegrown monitoring tools

2010s · Virtualized Deployment Era

- **Cloud computing** — VMs in minutes
- **Config management**: Puppet, Chef (infra as code)
- **Manual bin-packing** of apps onto VMs
- Better tooling, but **scale was still hard**

2020s · Container Deployment Era

- **Orchestrators** treat a cluster of machines as **one resource**
- Standard utilities for the whole lifecycle

What orchestrators standardised

🎯 Efficient scheduling

Place workloads optimally across instances.

❤️ Health checks

Monitor application health & status.

🔎 Service discovery

Automatically find services in the cluster.

📝 Configuration management

A standard way to manage & apply config.

📈 Autoscaling

Adjust instance counts to demand.

💾 Persistent storage

Storage that outlives containers.

🌐 + Networking

Reliable, scalable networking between services — the seventh pillar.

🎬 Go deeper

Honeypot's two-part "Kubernetes: The Documentary" on YouTube covers the origin story.

🌟 Key Takeaways — Chapter 1

- 2000s: bare metal, monoliths, sysadmins, on-prem.
- 2010s: cloud VMs, config management, manual bin-packing — scale still painful.
- 2020s: orchestrators treat a cluster as one resource with standard interfaces.
- Kubernetes brings scheduling, health checks, discovery, config, autoscaling, storage, networking.

✅ Quick-Review Checklist

- ✅ Name the three deployment eras
- ✅ Key traits of each era
- ✅ The 7 orchestrator capabilities
- ✅ Why scale became hard in the 2010s

📝 Practice Questions

Q1. **What defined the Traditional Deployment Era?**
→ On-prem/bare-metal servers, monoliths, sysadmin teams, homegrown monitoring.

Q2. **What innovation defined the 2010s?**
→ Cloud computing (VMs in minutes) plus config-management tools like Puppet/Chef.

Q3. **What is "manual bin-packing"?**
→ Hand-allocating applications onto VMs to optimise resource usage.

Q4. **What is the key promise of workload orchestrators?**
→ Treating a cluster of machines as a single resource with standard interfaces.

Q5. **Name three capabilities orchestrators standardised.**
→ Any of: scheduling, health checks, service discovery, config, autoscaling, storage, networking.

Kubernetes · Part 2

2 Technology Overview

Nodes, the control plane and data plane, and the system components that make a cluster.

2.1 Planes and nodes

🖥️ Node

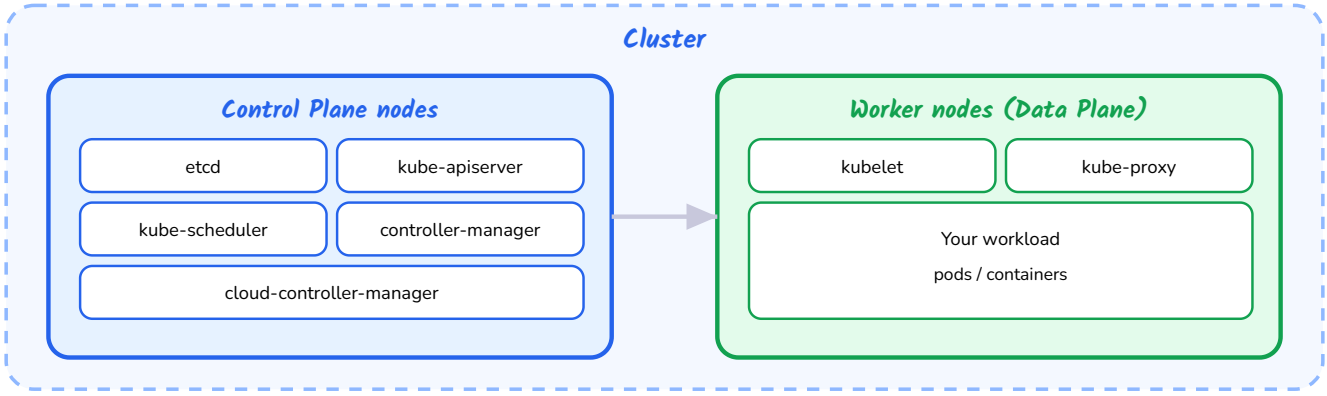
A computer / server. Several nodes joined together form a **cluster**.

🧭 Control plane

The subset of nodes that run **system tasks**. These are **control plane nodes**.

⚙️ Data plane

The subset of nodes that run **user workloads** — the **worker nodes**.



Control plane manages the cluster; worker nodes run your workloads.

2.2 Kubernetes system components

Component	Plane	Role
etcd	Control	Key-value store of **all cluster data** — the source of truth for state & config.
kube-apiserver	Control	Front end of the control plane; everything talks to it via the API.
kube-scheduler	Control	Places pods on the best node by resources & constraints.
kube-controller-manager	Control	Runs controller processes that drive resources to their desired state (replication, node ops...).
cloud-controller-manager	Control	Integrates with the cloud provider — load balancers, storage, networking.
kubelet	Data	Agent on each worker node; ensures containers in pods run and manages their lifecycle.
kube-proxy	Data	Network proxy on each node maintaining rules so pods can communicate.

🧠 A memory hook

etcd remembers · **apiserver** is the front door · **scheduler** picks a home · **controller-manager** keeps things right · **cloud-controller** bridges to the cloud · **kubelet** runs the containers · **kube-proxy** wires the network.

🌟 Key Takeaways — Chapter 2

- A **node** is a server; nodes form a **cluster**.
- **Control plane** nodes run system components; **worker nodes** run workloads.
- etcd is the source of truth; the API server is the single entry point.
- Scheduler places; controllers reconcile; kubelet executes; kube-proxy networks.

✅ Quick-Review Checklist

- ✅ Node vs cluster
- ✅ Control plane vs data plane
- ✅ Role of each of the 7 components
- ✅ Which components run on worker nodes

📝 Practice Questions

Q1. **What is the source of truth for cluster state?**
→ **etcd**.

Q2. **What is the front end of the control plane?**
→ **kube-apiserver**.

Q3. **Which component decides which node a pod runs on?**
→ **kube-scheduler**.

Q4. **What runs on each worker node to manage containers?**
→ **kubelet** (and **kube-proxy** for networking).

Q5. What does the cloud-controller-manager do?

→ Bridges Kubernetes with the cloud provider for load balancers, storage, and networking.

Q6. Difference between control plane and data plane?

→ Control plane runs system tasks; data plane (worker nodes) runs user workloads.

Kubernetes · Part 3

3 Installation & Setup

Devbox for tooling, plus three cluster flavours: KinD, Civo, and GKE.

3.1 Dependencies

🐳 Docker Desktop

Install per your OS. Needed for KinD (Kubernetes-in-Docker).

📦 Devbox

All other tools are pinned in `devbox.json` / `devbox.lock`. Run `devbox shell` from anywhere in the repo — Nix installs an isolated copy of everything.

⌨️ Aliases & completion

```
k=kubectl
t=task
tl='task --list-all'
```

Then set up tab completion for `kubectl` and `task`.

3.2 Three clusters you can use

Any of the three works for most examples — the exceptions are cloud-specific features and public-DNS exposure.

🐳 KinD (local)

Kubernetes **inside Docker**. Free, great for dev/testing. Can't demo cloud features or public DNS.

```
task kind:01-generate-config
task kind:02-create-cluster
```

☁️ Civo Cloud

Simple managed k8s; fast provisioning and a **1 month / $250** trial for new users.

```
civo:01-authenticate-cli
civo:04-create-all
civo:05-get-kubeconfig
# destroy: civo:06-clean-up
```

🔵 Google GKE

The longest-running managed k8s; **90-day / $300** free trial.

```
gcp:01-init-cli
gcp:07-create-all
gcp:08-connect-to-cluster
# destroy: gcp:09-clean-up
```

🚨 Clean up to avoid bills

Cloud clusters can leave behind load balancers, VPCs, and subnets that reference each other. You may need to **manually remove** those resources before teardown succeeds — and verify everything is gone to avoid surprise costs.

💡 Why Devbox + Task?

Devbox guarantees identical tool versions across machines. **Task** (`task kind:…`) wraps the multi-step setup commands so everyone runs the same thing.

🌟 Key Takeaways — Chapter 3

- Devbox + Nix gives reproducible tooling from `devbox.json`.
- **KinD** = free local k8s in Docker; good for nearly everything except cloud/DNS features.
- **Civo** and **GKE** = managed clusters with generous trials.
- `task` targets standardise cluster creation/teardown.
- Always clean up cloud resources to avoid cost.

✅ Quick-Review Checklist

- ✅ What Devbox provides
- ✅ Which cluster to use for cloud-specific demos
- ✅ Commands to create/destroy each cluster
- ✅ Cleanup caveats

📝 Practice Questions

Q1. **What does `devbox shell` do?**
→ Uses Nix to install an isolated, pinned copy of all required software.

Q2. **What is KinD?**
→ Kubernetes-in-Docker: a local cluster running inside Docker containers.

Q3. **Which cluster can't demo cloud-specific or public-DNS features?**
→ KinD.

Q4. **Which provider has run managed k8s the longest?**
→ Google (GKE).

Q5. **Why might cloud teardown fail?**
→ Leftover resources (load balancers, VPCs, subnets) referencing each other must be removed first.

4.1 Namespaces, Pods, ReplicaSets & Deployments

The building blocks: how to isolate, run, and manage workloads.

🧭 How these examples work

Each resource type has its own directory + Taskfile. Tasks are numbered in the order to run (`task 01-create-namespace` … `task 07-delete-namespace`). Every example lives in its own **namespace** so cleanup is just deleting the namespace.

4.1.1 Namespaces

🗂️ What is it?

A way to **divide cluster resources among multiple users/teams**. Names are unique *within* a namespace, not across the cluster. Good for isolation and easy cleanup.

📄 Namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: 04--namespace-file
```

4.1.2 Pods

📦 What is it?

The **smallest and simplest** Kubernetes object — a set of running containers on your cluster. Containers in a pod share network + storage. Pods are **ephemeral**: don't rely on them surviving.

🚨 Don't create bare Pods in prod

A standalone Pod won't restart or reschedule if it dies. Use a **Deployment** (which manages ReplicaSets → Pods) for real workloads.

📄 Pod.nginx-minimal.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-minimal
spec:
  containers:
    - name: nginx
      image: nginx:1.26.0
```

✅ Pod.nginx-better.yaml — production-grade

```yaml
spec:
  containers:
    - name: nginx
      image: cgr.dev/chainguard/nginx:latest   # hardened base
      ports: [{containerPort: 8080, protocol: TCP}]
      readinessProbe:                          # don't route traffic until ready
        httpGet: {path: /, port: 8080}
      resources:                               # requests + limits
        requests: {memory: "25Mi", cpu: "250m"}
        limits:   {memory: "50Mi"}
      securityContext:
        allowPrivilegeEscalation: false
        privileged: false
  securityContext:                             # pod-level security
    runAsNonRoot: true
    runAsUser: 1001
    runAsGroup: 1001
    seccompProfile: {type: RuntimeDefault}
```

4.1.3 ReplicaSets

🔁 What is it?

Ensures a specified number of **pod replicas** are running at all times. If a pod dies, the ReplicaSet creates a new one. It selects pods via **label selectors**.

📄 ReplicaSet.nginx-minimal.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-minimal
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-minimal       # ← must match the template labels
  template:
    metadata:
      labels:
        app: nginx-minimal
    spec:
      containers:
        - name: nginx
          image: nginx:1.26.0
```

🔗 The selector ↔ label contract

The ReplicaSet's `selector.matchLabels` must match the pod template's `metadata.labels`, otherwise it manages nothing (or hijacks unrelated pods).

4.1.4 Deployments

🚀 What is it?

Manages **stateless applications**. It owns ReplicaSets and adds **rolling updates** and **rollbacks**. This is the everyday resource for web apps.

📄 Deployment.nginx-minimal.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-minimal
spec:
  replicas: 3
  selector:
    matchLabels: {app: nginx-minimal}
  template:
    metadata:
      labels: {app: nginx-minimal}
    spec:
      containers:
        - name: nginx
          image: nginx:1.26.0
```

Deployment → ReplicaSet → Pods. You edit the Deployment; Kubernetes handles the rest.

🛠️ Everyday commands

```
kubectl apply -f Deployment.yaml
kubectl get pods -n 04--deployment
kubectl rollout status deployment/nginx-minimal -n 04--deployment
kubectl set image deployment/nginx-minimal nginx=nginx:1.27.0 -n 04--deployment
kubectl rollout undo deployment/nginx-minimal -n 04--deployment
```

🌟 Key Takeaways — 4.1

- **Namespaces** divide cluster resources; unique names within, easy cleanup.
- **Pods** = smallest unit (one or more containers sharing net/storage); ephemeral.
- **ReplicaSets** keep N pods running via label selectors.
- **Deployments** manage ReplicaSets with rolling updates & rollbacks — use for stateless apps.
- Selector labels must match template labels, or nothing works.

✅ Quick-Review Checklist

- Namespace purpose
- Why not use bare Pods
- Selector ↔ label relationship
- ReplicaSet vs Deployment
- Rollout / rollback commands
- Readiness probe, requests/limits, securityContext roles

📝 Practice Questions

Q1. Why are Namespaces useful?
→ They divide cluster resources for isolation and make cleanup a single delete.

Q2. What's the smallest deployable Kubernetes object?
→ The **Pod**.

Q3. What does a ReplicaSet guarantee?
→ That a specified number of pod replicas are running at all times.

Q4. What does a Deployment add over a ReplicaSet?
→ Rolling updates and rollbacks (versioned ReplicaSets).

Q5. What happens if selector labels don't match template labels?
→ The controller manages no pods (or the wrong ones).

Q6. Why set a readinessProbe?
→ Traffic shouldn't route to a pod until it's actually ready to serve.

Q7. Difference between requests and limits?
→ **requests** are guaranteed for scheduling; **limits** are the max allowed before throttling/kill.

Q8. How do you roll back a Deployment?
→ `kubectl rollout undo deployment/NAME`.

Kubernetes · Part 4.2

4.2 Services

A stable address for an ever-changing set of pods: ClusterIP, NodePort, LoadBalancer.

🌐 What is a Service?

Defines a **logical set of pods** and a policy to access them. It gives a stable DNS name / IP in front of pods that come and go, and load-balances across the selected pods.

Anatomy

Field	Meaning
`selector`	which pods this service routes to (by label)
`port`	the port the **service** listens on
`targetPort`	the **container** port (defaults to `port`)
`nodePort`	(NodePort) port on each node, 30000–32767 if unset
`type`	ClusterIP (default), NodePort, LoadBalancer

4.2.1 ClusterIP (the default)

📄 Service.nginx-clusterip.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-clusterip
  labels: {foo: service-label}
  annotations: {bar: service-annotation}
spec:
  type: ClusterIP          # default
  selector: {baz: pod-label}
  ports:
    - protocol: TCP
      port: 80             # service listens here
      targetPort: 80       # container listens here
```

*Reachable only **inside** the cluster. The standard way services talk to each other.*

4.2.2 NodePort

Service.nginx-nodeport.yaml

```
spec:
  type: NodePort
  selector: {baz: pod-label}
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      # nodePort: 30XXX  (auto-assigned 30000-32767 if omitted)
```

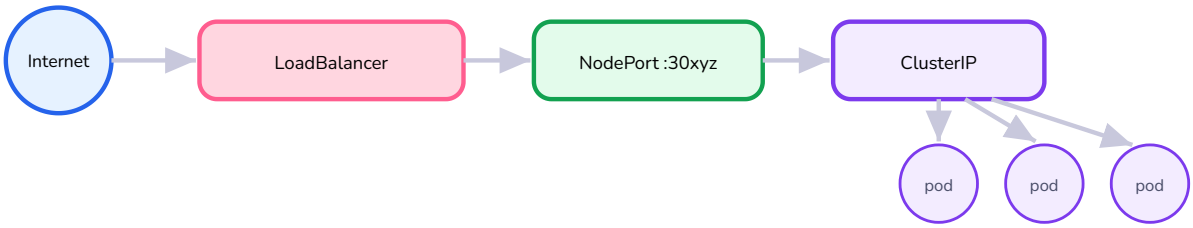
*Exposes the service on a port of **every node** — reachable from outside via nodeIP:nodePort.*

4.2.3 LoadBalancer

Service.nginx-loadbalancer.yaml

```
spec:
  type: LoadBalancer   # needs a cluster that can provision one (cloud provider)
  selector: {baz: pod-label}
  ports: [{protocol: TCP, port: 80, targetPort: 80}]
```

Requests a cloud load balancer with a public IP — the simplest way to expose a service externally.



ClusterIP is the core; NodePort exposes it on nodes; LoadBalancer fronts it with a cloud LB.

🔎 DNS

A Service gets a DNS name: `<service>.<namespace>.svc.cluster.local`. Within the same namespace, just the service name works — that's how pods discover each other.

🌟 Key Takeaways — 4.2

- Services give pods a stable address and load-balance across the selected set.

- **ClusterIP** = internal only (default). **NodePort** = each node's port. **LoadBalancer** = cloud LB.
- `port` is the service's port; `targetPort` is the container's.
- Discovery via DNS (service name within the namespace).

✅ Quick-Review Checklist

- [x] The three Service types
- [x] port vs targetPort vs nodePort
- [x] When LoadBalancer works
- [x] Service DNS format
- [x] What selector does

📝 Practice Questions

Q1. **Default Service type?**
→ ClusterIP.

Q2. **Difference between `port` and `targetPort`?**
→ `port` is what the service listens on; `targetPort` is the container port.

Q3. **Which type exposes a port on each node?**
→ NodePort (30000–32767 if unspecified).

Q4. **What does LoadBalancer require?**
→ A cluster/cloud that can provision an external load balancer.

Q5. **How does a pod in the same namespace reach a service called `db`?**
→ Just by hostname `db` (full form `db.<ns>.svc.cluster.local`).

Q6. **How does a Service find its pods?**
→ Via its `selector` matching pod labels.

Kubernetes · Part 4.3

4.3 Ingress & the Gateway API

HTTP routing into the cluster — the mature standard (Ingress) and its successor (Gateway API).

4.3.1 Ingress

What is it?

Manages **external HTTP(S) access** to services in the cluster. An **ingress controller** (nginx, GCE, Traefik…) implements the rules; the Ingress object just declares them (host + path → service).

Ingress.minimal-nginx.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-nginx
spec:
  ingressClassName: nginx
  rules:
    - host: "ingress-example-nginx.com"
      http:
        paths:
          - path: /
            pathType: ImplementationSpecific
            backend:
              service:
                name: nginx-clusterip
                port: {number: 80}
```

Ingress.fuller-nginx.yaml — controller-specific behaviour

```
metadata:
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "false"
    nginx.ingress.kubernetes.io/use-regex: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "50m"
spec:
  ingressClassName: nginx
  defaultBackend:
    service: {name: default-backend, port: {number: 80}}
  rules:
    - host: "civo-ingress-example.com"
      http:
        paths:
          - {path: /foo,          pathType: Prefix,              backend: {service: {name:
foo-service,    port: {number: 80}}}}
          - {path: /bar,          pathType: ImplementationSpecific, backend: {service: {name:
bar-service,  port: {number: 8080}}}}
          - {path: "/static/(.*)", pathType: ImplementationSpecific, backend: {service: {name:
static-service, port: {number: 80}}}}
```

Host + path routing, a default backend, and annotations that configure the nginx controller.

GKE quirk

On GKE you can't use `spec.ingressClassName` — select the controller with the annotation instead:

```
metadata:
  annotations:
    kubernetes.io/ingress.class: "gce"   # or "gce-internal"
# backend points at a NodePort service (e.g. nginx-nodeport)
```

4.3.2 The Gateway API

What is it?

A newer, **role-oriented** successor to Ingress with richer routing. It splits concerns into distinct resources: **GatewayClass** (infrastructure provider) → **Gateway** (a load balancer/listener) → **HTTPRoute** (routing rules, owned by app teams).

🏷️ GatewayClass.kong.yaml

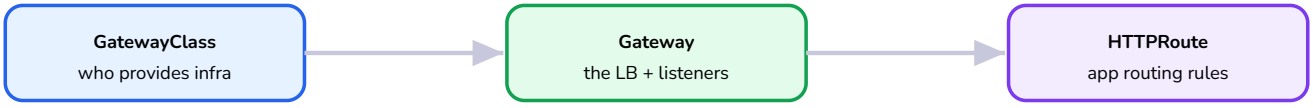
```
kind: GatewayClass
metadata: {name: kong}
spec:
  controllerName:
    konghq.com/kic-
gateway-controller
```

🚪 Gateway.kong.yaml

```
kind: Gateway
metadata: {name: kong}
spec:
  gatewayClassName: kong
  listeners:
    - {name: proxy,
port: 80, protocol:
HTTP}
```

🧭 HTTPRoute.kong.yaml

```
kind: HTTPRoute
metadata: {name: kong}
spec:
  parentRefs: [{name:
kong}]
  hostnames: ["gateway-
example-kong.com"]
  rules:
    - matches:
        - path: {type:
PathPrefix, value: /}
      backendRefs:
        - {name: nginx-
clusterip, kind:
Service, port: 80}
```



Gateway API separates infra (platform team) from routing (app team).

🌟 Key Takeaways — 4.3

- **Ingress** routes external HTTP(S) traffic to services; needs an **ingress controller**.
- Rules: `host` + `path` → service/port; `pathType` matters.
- Controller behaviour is tuned with **annotations** (e.g. rewrites, regex, ssl-redirect).
- **Gateway API** is the modern successor: GatewayClass → Gateway → HTTPRoute.
- GKE uses annotations for the ingress class, not `spec.ingressClassName`.

✅ Quick-Review Checklist

- ✅ Ingress vs ingress controller
- ✅ Anatomy of an Ingress rule
- ✅ Common nginx annotations
- ✅ GatewayClass / Gateway / HTTPRoute roles
- ✅ GKE ingress class quirk

📝 Practice Questions

Q1. Does an Ingress do anything on its own?
→ No — you need an **ingress controller** to implement the rules.

Q2. What maps traffic in an Ingress rule?
→ `host` + `path` → backend service & port.

Q3. How do you configure controller-specific behaviour?
→ Via `metadata.annotations` (e.g. `nginx.ingress.kubernetes.io/…`).

Q4. Name the three Gateway API resources here.
→ GatewayClass, Gateway, HTTPRoute.

Q5. What's the GKE Ingress gotcha?
→ You cannot use `spec.ingressClassName`; use the `kubernetes.io/ingress.class` annotation.

Q6. Why is Gateway API considered more flexible?
→ It's role-oriented (separate infra vs routing) and supports richer routing than Ingress.

Kubernetes · Part 4.4

4.4 Jobs, CronJobs & DaemonSets

Workloads that aren't long-running web servers: run-to-completion, scheduled, and per-node.

4.4.1 Jobs

✅ What is it?

Creates one or more pods that **run to completion** (batch work): migrations, data processing, one-off scripts.

📄 Job.echo-date-minimal.yaml

```yaml
apiVersion: batch/v1
kind: Job
metadata: {name: echo-date-minimal}
spec:
  template:
    spec:
      containers:
        - name: echo
          image: busybox:1.36.1
          command: ["date"]
      restartPolicy: Never     # required for Jobs
  backoffLimit: 1              # retries before marking failed
```

📄 Job.echo-date-better.yaml — key tuning fields

```yaml
spec:
  parallelism: 2               # run 2 pods at once
  completions: 2               # total successful completions needed
  activeDeadlineSeconds: 100   # hard time limit
  backoffLimit: 1
```

4.4.2 CronJobs

⏰ What is it?

Schedules **Jobs** at times/intervals using a familiar **cron** syntax. It wraps a `jobTemplate`.

📄 CronJob.echo-date-minimal.yaml

```yaml
apiVersion: batch/v1
kind: CronJob
metadata: {name: echo-date-minimal}
spec:
  schedule: "* * * * *"        # every minute
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - {name: echo, image: busybox:1.36.1, command: ["date"]}
          restartPolicy: Never
      backoffLimit: 1
```

🕑 The five cron fields

`minute hour day-of-month month day-of-week` — e.g. `* * * * *` = every minute, `0 3 * * *` = daily at 03:00.

4.4.3 DaemonSets

🧩 What is it?

Ensures a copy of a pod runs on **every (or some) node**. Use it for node-level agents: log collectors, metrics exporters, storage daemons, networking.

📄 DaemonSet.fluentd-minimal.yaml

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata: {name: fluentd-minimal, namespace: 04--daemonset}
spec:
  selector: {matchLabels: {app: fluentd}}
  template:
    metadata: {labels: {app: fluentd}}
    spec:
      containers:
        - {name: fluentd, image: fluentd:v1.16-1}
```

⚖️ Job vs Deployment

Job = finishes. Deployment = runs forever.

⚖️ CronJob vs Job

CronJob creates Jobs on a schedule.

⚖️ DaemonSet vs Deployment

DaemonSet = one pod per node (agent). Deployment = N pods anywhere.

🌟 Key Takeaways — 4.4

- **Job**: run-to-completion; needs `restartPolicy: Never|OnFailure`; tune `parallelism`, `completions`, `backoffLimit`, `activeDeadlineSeconds`.
- **CronJob**: schedules Jobs with cron syntax via `jobTemplate`.
- **DaemonSet**: one pod per node for node-level agents.

✅ Quick-Review Checklist

- When to use a Job
- parallelism / completions / backoffLimit
- Cron syntax fields
- DaemonSet use cases
- Required restartPolicy for Jobs

📝 Practice Questions

Q1. **Which resource runs pods to completion?**
→ A **Job**.

Q2. **What restartPolicy do Jobs require?**
→ `Never` or `OnFailure` (not `Always`).

Q3. **What does `backoffLimit` control?**
→ How many times the Job retries before it's marked failed.

Q4. **What is a CronJob?**
→ A scheduled Job, using cron syntax in a `jobTemplate`.

Q5. **Give two DaemonSet use cases.**
→ Log collection (e.g. fluentd) and metrics/node agents.

Q6. **How many pods does a DaemonSet create?**
→ One per eligible node.

Kubernetes · Part 4.5

4.5 StatefulSets & Persistent Storage

Stable identity for pods that hold state, plus PV / PVC / StorageClass.

4.5.1 StatefulSets

What is it?

Manages **stateful applications**, guaranteeing the **ordering and uniqueness** of pods. Pods get stable names (`nginx-0` , `nginx-1` , ...) and optionally their own storage.

StatefulSet.nginx-minimal.yaml

```
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: nginx-minimal}
spec:
  serviceName: nginxs     # the headless service giving each pod DNS
  replicas: 3
  selector: {matchLabels: {app: nginx}}
  template:
    metadata: {labels: {app: nginx}}
    spec:
      containers:
        - {name: nginx, image: nginx:1.26.0}
```

Stable network identity

A **headless service** (`clusterIP: None`) gives each replica its own DNS name: `nginx-0.nginxs.<ns>.svc.cluster.local` . That's why the service is named *plural* (`nginxs`) while the per-address service is singular (`nginx`).

initContainer + per-pod storage (excerpt)

```
spec:
  serviceName: nginxs
  replicas: 3
  template:
    spec:
      initContainers:                    # runs before the main container
        - name: populate-default-html
          image: nginx:1.26.0
          command:
            - bash
            - "-c"
            - |
              set -ex
              [[ $HOSTNAME =~ -([0-9]+)$ ]] || exit 1
              ordinal=${BASH_REMATCH[1]}
              echo "Hello from pod $ordinal" > /usr/share/nginx/html/index.html
          volumeMounts:
            - {name: data, mountPath: /usr/share/nginx/html}
      containers:
        - name: nginx
          image: nginx:1.26.0
          volumeMounts:
            - {name: data, mountPath: /usr/share/nginx/html}
  volumeClaimTemplates:                  # a PVC per replica (data-nginx-0, data-nginx-1 …)
    - metadata: {name: data}
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: "standard"
        resources: {requests: {storage: 100Mi}}
```

*Each replica parses its **ordinal** from its hostname and writes pod-specific content — e.g. a DB cluster where some pods are read/write and others read-only.*

4.5.2 Persistent storage: PV / PVC / StorageClass

PersistentVolume (PV)

The actual piece of storage. Can be provisioned **manually** (admin) or **dynamically** (via StorageClass).

PersistentVolumeClaim (PVC)

An application's **request** for storage. Pods mount a PVC, not a PV directly.

StorageClass

A **provisioner recipe** for creating PVs on demand. Marked default via an annotation.

📄 StorageClass.standard.yaml

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: standard
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: rancher.io/local-path        # sets up dynamic provisioning
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer   # bind only when a pod needs it
```

⚡ Dynamic PVC

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: dynamic-pv-kind}
spec:
  accessModes: [ReadWriteOnce]
  resources: {requests: {storage: 100Mi}}
  storageClassName: standard
```

No matching PV needed — the StorageClass creates one.

🔧 Manual PV + selector

```yaml
kind: PersistentVolume
spec:
  accessModes: [ReadWriteOnce]
  capacity: {storage: 100Mi}
  storageClassName: standard
  local: {path: /some/path/in/container}
  nodeAffinity:                # local
PVs are tied to a node
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - {key:
kubernetes.io/hostname,
              operator: In, values:
[kind-worker]}
# PVC then selects it:
  selector: {matchLabels: {name: manual-
kind}}
```

🔒 accessModes matter

`ReadWriteOnce` = one node can mount read/write, so replicas may be constrained to the same node (as the deck's `shared-pvc-kind` Deployment notes). Multi-node sharing needs other modes.

🌟 Key Takeaways — 4.5

- **StatefulSet** gives stable, ordered pod identities and per-pod storage.
- Headless Service (`clusterIP: None`) provides per-pod DNS names.
- **PVC** = request; **PV** = actual storage; **StorageClass** = dynamic provisioner.
- `volumeClaimTemplates` creates a PVC per replica.
- `initContainers` run before the app container for setup.

- `WaitForFirstConsumer` + local PVs bind to a node.

✅ Quick-Review Checklist

- StatefulSet vs Deployment
- What makes a service "headless"
- PV vs PVC vs StorageClass
- Dynamic vs manual provisioning
- volumeClaimTemplates purpose
- Reclaim policy & binding mode

📝 Practice Questions

Q1. What guarantees does a StatefulSet provide?
→ Stable, unique pod names/ordering and stable storage per pod.

Q2. How do you give each replica its own DNS name?
→ With a headless Service (`clusterIP: None`) referenced by `serviceName`.

Q3. PVC vs PV?
→ PVC is the app's request; PV is the actual storage bound to it.

Q4. What enables dynamic provisioning?
→ A StorageClass with a provisioner.

Q5. What does `volumeClaimTemplates` do?
→ Creates a separate PVC for each StatefulSet replica.

Q6. What is an initContainer?
→ A container that runs to completion before the app containers start (setup, seeding).

Kubernetes · Part 4.6

4.6 ConfigMaps & Secrets

Decouple configuration and sensitive data from your container images.

4.6.1 ConfigMaps

🗒️ What is it?

Stores **non-sensitive configuration** that pods consume — as environment variables or mounted files.

🔑 Property-like keys (env vars)

```
apiVersion: v1
kind: ConfigMap
metadata: {name: property-like-keys}
data:
  NAME: YourAppName
  VERSION: 1.0.0
  AUTHOR: YourName
```

📄 File-like keys (mounted files)

```
apiVersion: v1
kind: ConfigMap
metadata: {name: file-like-keys}
data:
  conf.yml: |
    name: YourAppName
    version: 1.0.0
    author: YourName
```

📄 Consuming both in a Pod

```
spec:
  containers:
    - name: nginx
      image: nginx:1.26.0
      volumeMounts:                # file-like → files under /etc/config
        - {name: configmap-file-like-keys, mountPath: /etc/config}
      envFrom:                     # property-like → env vars
        - configMapRef: {name: property-like-keys}
  volumes:
    - name: configmap-file-like-keys
      configMap: {name: file-like-keys}
```

Two consumption styles: `envFrom` / `valueFrom` *for env, volumes for files.*

4.6.2 Secrets

🔐 What is it?

Manages **sensitive** data — passwords, tokens, SSH keys, registry credentials. Secrets are only **base64-encoded, not encrypted**, by default — so enable encryption-at-rest in real clusters.

✍️ stringData (you write plaintext)

```
kind: Secret
type: Opaque
stringData:
  foo: bar
```

🔢 data (base64-encoded)

```
kind: Secret
type: Opaque
data:
  foo: YmFy      # echo -n bar | base64
```

📄 Consuming a Secret as env + file + imagePullSecret

```
spec:
  containers:
    - name: nginx
      image: nginx:1.26.0
      volumeMounts:
        - {name: secret-base64-data, mountPath: /etc/config}
      env:
        - name: ENV_VAR_FROM_SECRET
          valueFrom:
            secretKeyRef: {name: base64-data, key: foo}
  imagePullSecrets:              # credentials to pull a private image
    - name: dockerconfigjson
  volumes:
    - name: secret-base64-data
      secret: {secretName: base64-data}
```

🐋 dockerconfigjson

A special Secret type (`kubernetes.io/dockerconfigjson`) holding registry credentials — referenced via `imagePullSecrets` so a pod can pull from a private registry.

🚨 Secrets ≠ encrypted by default

Anyone with read access to the Secret/etcd can decode base64 instantly. Treat permissions tightly (RBAC §4.7) and enable encryption at rest / external secret managers for real workloads.

🌟 Key Takeaways — 4.6

- **ConfigMap** = non-sensitive config; **Secret** = sensitive data.

- Both can be consumed as **env vars** or **mounted files**.
- `stringData` takes plaintext; `data` expects base64.
- Secrets are base64-encoded, not encrypted — restrict access.
- `imagePullSecrets` uses a `dockerconfigjson` Secret for private registries.

✅ Quick-Review Checklist

- ConfigMap vs Secret
- Env vs file consumption
- stringData vs data
- secretKeyRef vs envFrom
- imagePullSecrets purpose

📝 Practice Questions

Q1. **ConfigMap vs Secret?**
→ ConfigMap for non-sensitive config; Secret for sensitive data.

Q2. **Two ways to consume them?**
→ As environment variables or as mounted files.

Q3. **Are Secrets encrypted by default?**
→ No — only base64-encoded. Enable encryption at rest / use an external manager.

Q4. **How do you pull from a private registry?**
→ Create a `dockerconfigjson` Secret and reference it via `imagePullSecrets`.

Q5. **Which field lets you write plaintext in a Secret?**
→ `stringData`.

Q6. **What does `envFrom.configMapRef` do?**
→ Injects every key of the ConfigMap as an environment variable.

Kubernetes · Part 4.7

4.7 RBAC & Pod SecurityContext

Who can do what (RBAC) and what a pod is allowed to do (securityContext).

4.7.1 RBAC — Role-Based Access Control

The model

Four objects. A **Role/ClusterRole** defines *permissions*; a **RoleBinding/ClusterRoleBinding** attaches them to a **subject** (a user or a **ServiceAccount**).

Namespaced (Role + RoleBinding)

Grants permissions **within one namespace**.

Cluster-wide (ClusterRole + ClusterRoleBinding)

Grants permissions **across the whole cluster**.

ServiceAccount

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: namespaced-pod-
reader
```

Role (permissions)

```
kind: Role
metadata: {name: pod-
reader}
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs:
["get","list","watch"]
```

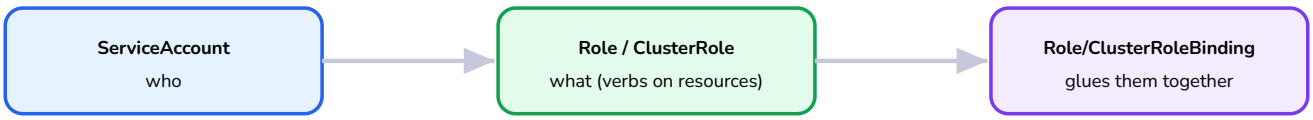
RoleBinding (attach)

```
kind: RoleBinding
metadata: {name: pod-
reader}
roleRef:
  kind: Role
  name: pod-reader
subjects:
  - kind: ServiceAccount
    name: namespaced-
pod-reader
```

⚙️ A Job that uses the ServiceAccount

```
kind: Job
spec:
  template:
    spec:
      serviceAccountName: namespaced-pod-reader   # ← uses the SA's permissions
      automountServiceAccountToken: true
      containers:
        - name: kubectl
          image: cgr.dev/chainguard/kubectl
          args: ["get", "pods", "-n", "04--rbac"] # namespaced → succeeds
      restartPolicy: Never
```

*The `cluster-pod-reader` variant runs `get pods -A` and succeeds via a ClusterRole. A Job with **no** ServiceAccount fails for lack of permission — a great "deny" demo.*



*Ask three questions: **who** (subject), **what** (rules), **where** (binding scope).*

4.7.2 Pod SecurityContext

🛡️ What is it?

Controls the **security settings** for a pod and its containers — the runtime equivalent of the Docker hardening flags from the Docker course (Ch. 9). It can be set at pod level and/or container level.

📄 Pod-level securityContext

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1001
  runAsGroup: 1001
  seccompProfile:
    type: RuntimeDefault
```

📄 Container-level securityContext

```
securityContext:
  allowPrivilegeEscalation: false
  privileged: false
  # readOnlyRootFilesystem: true
  # capabilities: {drop: ["ALL"]}
```

💡 Best-practice combo

Run as non-root, drop all capabilities, disable privilege escalation, use the default seccomp profile, and make the root filesystem read-only where possible. Together with hardened base images (Chainguard), these make a pod a very unattractive target.

🌟 Key Takeaways — 4.7

- RBAC = **Role/ClusterRole** (what) + **Binding** (attach) + **ServiceAccount** (who).
- Role/Binding = namespaced; ClusterRole/ClusterBinding = cluster-wide.
- Pods act with their ServiceAccount's permissions; no SA = minimal/no access.
- **securityContext** hardens pods: non-root, no privilege escalation, seccomp, dropped caps.

✅ Quick-Review Checklist

- ✅ The four RBAC objects
- ✅ Namespaced vs cluster-wide
- ✅ How a pod gets permissions
- ✅ Pod vs container securityContext
- ✅ Common hardening settings

📝 Practice Questions

Q1. **Name the four RBAC objects.**
→ Role, ClusterRole, RoleBinding, ClusterRoleBinding (plus ServiceAccount as the subject).

Q2. **Difference between Role and ClusterRole?**
→ Role applies within a namespace; ClusterRole applies cluster-wide.

Q3. **How does a Job/pod obtain permissions?**
→ Via its `serviceAccountName`, bound to roles through a binding.

Q4. **What happens with no ServiceAccount permissions?**
→ API calls are denied — the "no-permissions" Job fails.

Q5. **What does `runAsNonRoot: true` do?**
→ Prevents the container from starting as root (fails if the image defaults to root).

Q6. **What does `allowPrivilegeEscalation: false` prevent?**
→ Processes gaining more privileges than their parent (e.g. via setuid).

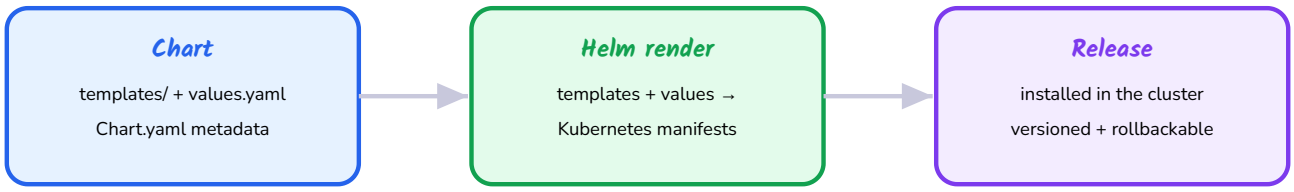
Kubernetes · Part 5

5 Helm

The de facto standard for packaging & distributing Kubernetes applications.

What is Helm?

Assembles **many Kubernetes resources** into a single package (**chart**) and uses **Go templating** to expose a custom interface, so consumers can configure the install for their needs. Popular ≠ only way (see Part 12 for Kustomize/Kluctl) — but it's common enough to be essential.



*A chart + values = rendered manifests, installed as a versioned **release**.*

5.1 Third-party charts (Bitnami Postgres)

Most of the time you consume someone else's chart. Typical actions:

Command	What it does
`helm repo add bitnami …`	Add a chart repository locally
`helm search repo bitnami/postgresql --versions`	Find charts/versions in a repo
`oras repo tags …`	List tags in an OCI-compliant registry
`helm pull bitnami/postgresql --version=…`	Download a chart locally
`helm show values …`	Show a chart's default values
`helm install postgresql bitnami/postgresql …`	Install a release
`helm upgrade --install …`	Upgrade (or install if missing)
`helm rollback postgresql`	Roll back to a previous revision
`helm list -n NS`	List releases
`helm get values / manifest …`	Inspect what's deployed
`helm uninstall postgresql`	Remove the release

install / upgrade / rollback

```
helm install postgresql bitnami/postgresql \
  --version=15.4.1 --create-namespace --values=values.yaml

helm upgrade --install postgresql bitnami/postgresql \
  --version=15.4.2 --create-namespace --values=values.yaml

helm rollback postgresql
```

`--version` pins the chart version; `--values` supplies your overrides.

5.2 First-party charts

Scaffold with `helm create`

```
helm create <CHART_NAME>
```

Boilerplate chart (Deployment, Service, Ingress, HPA, SA, tests…). Replace the files in `templates/` with your own. The repo keeps an unmodified copy for observation.

📄 The minimal chart's templating features

It deploys one ConfigMap but demonstrates nearly everything:

- variables (`define`)
- conditionals (`if`)
- foreach (`range`)
- `.Release` · `.Chart` · `.Values`

```
# templates/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: helm-{{ .Release.Name }}-configmap    # .Release
data:
  appVersion: {{ .Chart.AppVersion }}         # .Chart
  environment: {{ .Values.environment | quote }}   # .Values
  envShort: {{ template "envShort" . }}       # define + template
  {{- range .Values.configData }}             # range
  {{- if .enabled }}                          # if
  {{ .key }}: {{ .value | quote }}
  {{- end }}
  {{- end }}
```

```
# templates/_helpers.tpl  (named template = variables)
{{- define "envShort" -}}
{{- if eq .Values.environment "production" -}}
prod
{{- else -}}
non-prod
{{- end -}}
{{- end -}}
```

```
# values.yaml
environment: production
configData:
  - {key: conditionalKey, value: "this will be included", enabled: true}
```

🔀 Same chart, different values

```
helm upgrade --install minimal ./minimal                      # default values
helm upgrade --install minimal ./minimal --values=values-alt.yaml  # staging
```

`values-alt.yaml` sets `environment: staging` and `enabled: false`, so the conditional key disappears from the ConfigMap — one chart, many environments.

🌟 Key Takeaways — Chapter 5

- Helm packages many resources into a **chart**; installing one creates a **release**.
- Go templating + `values.yaml` gives a configurable interface.
- Know the lifecycle: repo add → search → pull/show → install → upgrade → rollback → list → uninstall.
- `helm upgrade --install` is the idempotent "apply" you'll use most.
- `helm create` bootstraps a chart; common features are define/if/range and the .Release/.Chart/.Values objects.

✅ Quick-Review Checklist

- ✅ Chart vs release
- ✅ helm repo / search / show / pull
- ✅ install vs upgrade --install
- ✅ rollback & list
- ✅ Templating: .Values, .Release, .Chart, define, if, range
- ✅ values overrides

—

📝 Practice Questions

Q1. **Chart vs release?**
→ A chart is the package; a release is an installed instance of it in a cluster.

Q2. **Which command installs-or-upgrades safely?**
→ `helm upgrade --install`.

Q3. **How do you see a chart's default values?**
→ `helm show values <chart>`.

Q4. **How do you roll back a release?**
→ `helm rollback <release>`.

Q5. **Which templating constructs does the minimal chart use?**
→ define (variables), if (conditionals), range (foreach), plus .Release/.Chart/.Values.

Q6. **How do you configure a chart for staging vs production?**
→ Pass different `--values` files to the same chart.

Kubernetes · Part 6

6 Demo Application

The 3-tier app from the Docker course, plus a load generator and request tracking.

Same app family as before — now the deployment target changes to Kubernetes. The additions make scaling and traffic visible.

React frontend

Uses react-query to call **both** APIs and display results.

Node & Go APIs

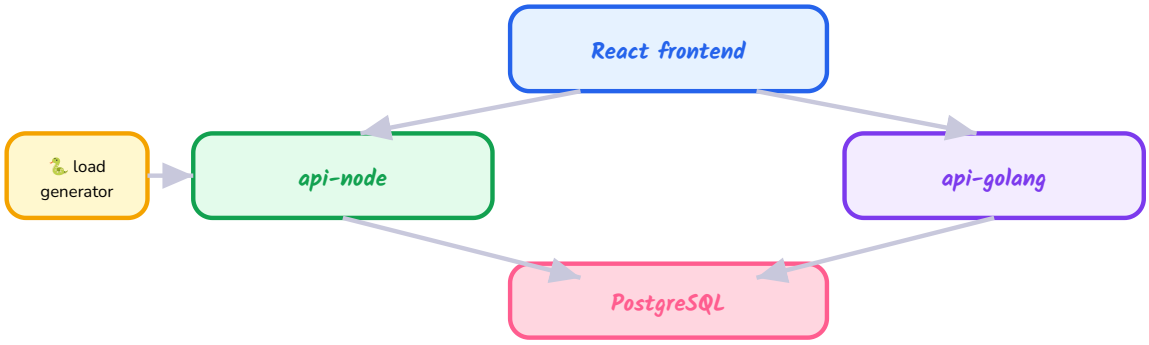
`/` returns the current time **and the per-API request counts**; `/ping` returns `pong`.

Postgres

Empty DB; APIs run `SELECT NOW() as now;`. A schema tracks requests per API.

Python load generator

Queries the Node **or** Go API at a configurable speed — perfect for watching scaling in action.



Load generator drives one API; both APIs read/write counts in Postgres; the client shows the result.

Running it locally (baseline)

Tier	Prep	Run
Postgres	—	`task postgresql:run-postgres`
DB schema	run migration	`task postgresql:run-psql-init-script` 🚨
api-node	`task api-node:install`	`task api-node:run` (nodemon)
api-golang	`task api-golang:install`	`task api-golang:run`
client-react	`task client-react:install`	`task client-react:run` (vite)
load-gen	`task load-generator-python:install`	`task load-generator-python:run`

🚨 Don't forget the migration

After starting Postgres you **must** run the migration in `./postgresql/migrations` to create the table the APIs use — otherwise `/` will fail.

🌟 Key Takeaways — Chapter 6

- Same 3-tier app as the Docker course, now aimed at Kubernetes.
- New: a **Python load generator** and a **request-counting** DB schema.
- Running locally now uses `task` targets instead of `make`.
- Postgres starts in a container; the migration is a required extra step.

✅ Quick-Review Checklist

- The four services
- What each API endpoint returns
- The extra DB schema's purpose
- Install/run task per service
- The migration gotcha

📝 Practice Questions

Q1. **What does the load generator do?**
→ Queries the Node or Go API at a configurable rate to generate traffic.

Q2. **What's new in this demo vs the Docker course version?**
→ A Python load generator and a DB schema that tracks requests per API.

Q3. **What does `/` now return?**
→ The current time plus per-API request counts.

Q4. **What must you run after starting Postgres locally?**
→ The migration script (`postgresql:run-psql-init-script`).

Q5. **Why is the load generator useful in this course?**
→ It creates real traffic so scaling/HPA/observability behaviour is visible.

7 Deploying the Demo Application

Turning the app into real manifests: Deployments, Services, IngressRoutes, Secrets, Jobs, Helm.

7.1 What we deploy

📦 The app services

The services from Part 6, described with k8s manifests.

🐘 PostgreSQL

Installed via the Bitnami **Helm chart**.

🚦 Traefik

Ingress controller (Helm), with routes for the app.

🗂️ Layout

```
api-golang/   Deployment · Service · IngressRoute · Secret
api-node/     Deployment · Service · IngressRoute · Secret
client-react/ Deployment · Service · IngressRoute · ConfigMap
common/       Namespace · Middleware
load-generator-python/ Deployment · ConfigMap
postgresql/   Job.db-migrator · Secret.db-password
```

7.2 The repeated pattern

Each service becomes four objects:

Deployment — runs the app → **Secret — DB credentials** → **Service — internal LB** → **IngressRoute — routes external traffic**

🏗️ And the migration is a Job

The DB migration runs as a Kubernetes Job, so it happens exactly once, to completion.

7.3 Key manifests

🔌 api-node/Deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: api-node, namespace: demo-app}
spec:
  replicas: 1
  selector: {matchLabels: {app: api-node}}
  template:
    metadata: {labels: {app: api-node}}
    spec:
      containers:
        - name: api-node
          image: sidpalas/…-api-node:foobarbaz
          env: [{name: PORT, value: "3000"}]
          envFrom:                      # DB URL from a Secret
            - secretRef: {name: api-node-database-url}
          ports: [{containerPort: 3000, protocol: TCP}]
          readinessProbe: {httpGet: {path: /ping, port: 3000}}
          resources:
            requests: {memory: "100Mi", cpu: "50m"}
            limits: {memory: "100Mi"}
          securityContext: {allowPrivilegeEscalation: false, privileged: false}
      securityContext: {seccompProfile: {type: RuntimeDefault}}
```

🌐 Service (ClusterIP)

```
kind: Service
metadata: {name: api-node, namespace:
demo-app}
spec:
  selector: {app: api-node}
  ports:
    - {protocol: TCP, port: 3000,
targetPort: 3000}
```

🔐 Secret (DB URL)

```
# ⛔ DONT PUT SECRET FILES IN VCS
kind: Secret
type: Opaque
metadata: {name: api-node-database-url,
namespace: demo-app}
stringData:
  DATABASE_URL:
postgres://postgres:foobarbaz@
    postgres-
postgresql.postgres.svc.cluster.local:543
2/postgres
```

🚦 Traefik IngressRoute + Middleware

```
kind: IngressRoute
metadata: {name: api-node, namespace: demo-app}
spec:
  entryPoints: [web]
  routes:
    - kind: Rule
      match: Host(`kubernetes-course.devopsdirective.com`) && PathPrefix(`/api/node`)
      middlewares: [{name: strip-api-prefixes}]
      services: [{kind: Service, name: api-node, port: 3000, scheme: http}]
---
kind: Middleware
metadata: {name: strip-api-prefixes, namespace: demo-app}
spec:
  stripPrefix:
    forceSlash: false
    prefixes: [/api/node, /api/golang]
```

Host + path matching, with a middleware that strips `/api/*` before forwarding.

🗄️ client-react/ConfigMap.yaml — nginx config as data

```
kind: ConfigMap
metadata: {name: nginx-conf, namespace: demo-app}
data:
  default.conf: |-
    server {
      listen 8080;
      location /ping { return 200 "pong"; }   # health endpoint
      location / {
        root /usr/share/nginx/html;
        try_files $uri $uri/ /index.html =404;  # SPA routing
      }
    }
```

Mounted into the pod at `/etc/nginx/conf.d`, replacing the default config.

🐍 load-generator-python/Deployment.yaml

```
env:
  - {name: API_URL, value: "http://api-node.demo-app.svc.cluster.local:3000"}
  - {name: DELAY_MS, value: "500"}
imagePullSecrets: [{name: dockerconfigjson}]   # private registry creds
```

*Note the **in-cluster DNS** name and the configurable pacing. A ConfigMap variant exists but is unused.*

🗃️ postgresql/Job.db-migrator.yaml

```
kind: Job
metadata: {name: db-migrator, namespace: demo-app}
spec:
  template:
    spec:
      containers:
        - name: migrate
          image: sidpalas/…-db-migrator:foobarbaz
          args:
            - -path=/app/migrations
            - -database=$(DATABASE_URL)?sslmode=disable
            - up
          envFrom: [{secretRef: {name: db-password}}]
      restartPolicy: OnFailure
```

A migration tool (goose-style) runs as a Job, reading creds from a Secret.

7.4 Deploying

⚙️ High-level tasks

```
task common:apply-namespace
task common:deploy-traefik
task common:apply-traefik-middleware
task postgresql:install-postgres
task postgresql:apply-initial-db-migration-job
task   api-node:apply / api-golang:apply / client-react:apply
task load-generator-python:apply
# or: task apply-all
```

🌟 Key Takeaways — Chapter 7

- Each service = **Deployment + Secret + Service + IngressRoute**.
- Postgres & Traefik come from **Helm**; the app comes from raw manifests.
- DB migration runs as a **Job** (run-to-completion).
- Config (nginx conf, load-gen env) lives in **ConfigMaps/Secrets**.
- Traefik **Middleware** strips `/api/*` prefixes; in-cluster DNS uses `svc.cluster.local`.

✅ Quick-Review Checklist

- ✅ The 4 objects per service
- ✅ Why the migration is a Job
- ✅ What Helm installs here
- ✅ IngressRoute match + middleware

- ✅ In-cluster DNS naming
- ✅ imagePullSecrets use

📝 Practice Questions

Q1. **Name the four objects each service gets.**
→ Deployment, Secret, Service, IngressRoute.

Q2. **How are Postgres and Traefik installed?**
→ Via Helm charts.

Q3. **Why is the DB migration a Job?**
→ It should run once to completion, then stop.

Q4. **What does the strip-api-prefixes Middleware do?**
→ Removes the `/api/node` and `/api/golang` prefixes before forwarding to services.

Q5. **How does the load generator reach the API?**
→ Via in-cluster DNS: `http://api-node.demo-app.svc.cluster.local:3000`.

Q6. **Where does the API get its DATABASE_URL?**
→ From a Secret via `envFrom.secretRef`.

Kubernetes · Part 8

8 Extending Kubernetes

CustomResourceDefinitions & Operators — turning "Declarative State + Continuous Reconciliation" into anything.

🧠 The core idea

Kubernetes ships with the resources needed to run containers. But the pattern **Declarative State + Continuous Reconciliation** is so powerful it can model almost anything. Rather than bake in every use case, the authors let anyone **extend the API**.

🔧 How the extension works

1. Authors create **CustomResourceDefinitions (CRDs)** — Kubernetes then stores those resources in **etcd**.
2. A **Controller / Operator** watches the custom resources and acts to bring reality in line with the declared state.

8.1 Extend existing capabilities (Traefik)

🛰️ Traefik adds its own resources

```
k api-resources | grep traefik.containo.us/v
# ingressroutes, ingressroutetcps, ingressrouteudps,
# middlewares, middlewaretcps, serverstransports,
# tlsoptions, tlsstores, traefikservices …
```

*These go **beyond** the official Ingress spec — the IngressRoute and Middleware we used in Part 7.*

8.2 Embed operator logic (CloudNativePG)

Stateful apps need carefully sequenced operations (data integrity) that are hard to encode with StatefulSets alone. An **Operator** encodes that logic — doing what a human operator used to do.

🐘 CloudNativePG's CRDs

```
k api-resources | grep postgres
# backups, clusterimagecatalogs, clusters,
# imagecatalogs, poolers, scheduledbackups
```

Then a user deploys a full PostgreSQL cluster with a simple interface — plus backups and pooling. (Covered next in Part 9.)

8.3 Deploy external infrastructure (Crossplane)

☁️ Operators beyond the cluster

Crossplane is Infrastructure-as-Code (like Terraform) that **runs inside Kubernetes**. Custom resources map to external cloud resources, and the Crossplane app deploys/manages them. Because it's a control loop, you get automatic **drift detection & remediation** for free.

1️⃣ Extend capabilities

New resource kinds alongside built-ins (Traefik's IngressRoute).

2️⃣ Embed operator logic

Encode complex, sequenced operations (CloudNativePG).

3️⃣ Manage external infra

CRDs ↔ cloud resources, reconciled continuously (Crossplane).

🌟 Key Takeaways — Chapter 8

- The pattern **Declarative State + Continuous Reconciliation** generalises far beyond containers.
- **CRDs** register new resource types; k8s stores them in etcd.
- **Operators/Controllers** reconcile custom resources toward the declared state.
- Three flavours: extend capabilities (Traefik), embed operator logic (CloudNativePG), manage external infra (Crossplane).
- `kubectl api-resources` lists everything, including new CRDs.

✅ Quick-Review Checklist

- ✅ What a CRD is
- ✅ What an Operator does
- ✅ Where custom resources are stored
- ✅ Three extension use cases
- ✅ `api-resources` command

📝 Practice Questions

Q1. **What does a CRD do?**
→ Defines a custom resource type that Kubernetes can store (in etcd) and manage.

Q2. **What is an Operator?**
→ A controller that reconciles custom resources toward their declared state, encoding operational logic.

Q3. **Give an example of extending existing capabilities.**
→ Traefik's IngressRoute / Middleware CRDs beyond the Ingress spec.

Q4. **Which operator deploys PostgreSQL clusters with backups/pooling?**
→ CloudNativePG.

Q5. **What does Crossplane let you do?**
→ Manage external cloud infrastructure via CRDs, with automatic drift detection/remediation.

Q6. How do you list available resource types (incl. CRDs)?

→ `kubectl api-resources`.

9 Auxiliary Tooling

Operators & add-ons that make a cluster production-worthy: CloudNativePG, Trivy Operator, cert-manager.

9.1 CloudNativePG

🐘 What is it?

Makes deploying/operating **PostgreSQL clusters** on Kubernetes far easier. It **extends the API with CRDs** (`Cluster`, `Backup`, `ScheduledBackup`, `Pooler`...) and encodes the operational logic in an operator that runs in the cluster.

📄 Minimal cluster

```
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata: {name: cnpg-minimal}
spec:
  instances: 2
  storage: {size: 1Gi}
```

💾 Cluster with object-store backups (Civo example)

```
spec:
  instances: 2
  storage: {size: 1Gi}
  backup:
    barmanObjectStore:
      destinationPath: "s3://cnpg-backups"
      endpointURL: "https://objectstore.nyc1.civo.com"
      s3Credentials:
        accessKeyId:     {name: civo-object-store-creds, key: AWS_ACCESS_KEY_ID}
        secretAccessKey: {name: civo-object-store-creds, key: AWS_SECRET_ACCESS_KEY}
    retentionPolicy: "30d"
```

Backups go to Google Cloud Storage or Civo object storage. Retention is 30 days.

⏰ ScheduledBackup

```
kind: ScheduledBackup
metadata: {name: daily}
spec:
  schedule: "0 0 0 * * *"     # six
fields, incl. seconds
  backupOwnerReference: self
  cluster: {name: cnpg-with-backup-
config}
```

📸 On-demand Backup

```
kind: Backup
metadata: {name: now-0}
spec:
  method: barmanObjectStore
  cluster: {name: cnpg-with-backup-
config}
```

✸ Install the operator (Helm)

```
helm repo add cnpg https://cloudnative-pg.github.io/charts
helm upgrade --install cnpg \
  --namespace cnpg-system --create-namespace \
  cnpg/cloudnative-pg
```

9.2 Trivy Operator

🔎 What is it?

Automatically **scans every container image** running in the cluster and produces reports of CVEs and other configuration risks. It **re-scans at intervals**, catching vulnerabilities discovered *after* the image was built — complementing CI-time scans.

✸ Install

```
helm repo add aqua https://aquasecurity.github.io/helm-charts/
helm repo update
helm upgrade --install trivy-operator aqua/trivy-operator \
  --namespace trivy-system --create-namespace \
  --version 0.23.2
```

9.3 cert-manager

🔐 What is it?

Provisions and manages **TLS certificates** for encrypted traffic to your cluster/services. Integrates with certificate authorities like **Let's Encrypt** and with Ingress controllers to automate issuance/renewal.

Not implemented in this repo, but important enough to mention — a great exercise.

🐘 CloudNativePG

DB clusters + backups as code.

🔎 Trivy Operator

Continuous in-cluster image scanning.

🔐 cert-manager

Automated TLS certificates.

🌟 Key Takeaways — Chapter 9

- **CloudNativePG**: PostgreSQL clusters via CRDs, with object-store backups (Backup/ScheduledBackup) and pooling.
- **Trivy Operator**: continuously scans in-cluster images; re-scans to catch newly-discovered CVEs.
- **cert-manager**: automates TLS certificates with Let's Encrypt + Ingress controllers.
- All three install via Helm and demonstrate the Operator/CRD pattern in action.

✅ Quick-Review Checklist

- ✅ CNPG CRDs and backup config
- ✅ Backup vs ScheduledBackup
- ✅ What Trivy Operator scans & when
- ✅ What cert-manager automates
- ✅ How each is installed

📝 Practice Questions

Q1. **What does CloudNativePG give you?**
→ Managed PostgreSQL clusters on k8s with backups, retention, and pooling via CRDs.

Q2. **Which CNPG resource schedules backups?**
→ `ScheduledBackup` (with a six-field cron schedule).

Q3. **Where can CNPG backups go?**
→ An object store (e.g. Google Cloud Storage or Civo object storage) via `barmanObjectStore`.

Q4. **What does Trivy Operator do that a CI scan doesn't?**
→ It re-scans running images periodically, catching CVEs found after build.

Q5. **What is cert-manager for?**
→ Provisioning/renewing TLS certificates, integrated with Let's Encrypt and ingress controllers.

Q6. **How are these tools installed?**
→ Via Helm charts.

Kubernetes · Part 10

10 Developer Experience

Fast inner loops (Tilt) and sane secrets management (External Secrets Operator).

10.1 The inner development loop

🔁 Goal

A dev environment that mirrors production as closely as possible. With plain KinD, any code change needs a **manual image rebuild** — painful. Tools fix this.

🎛️ Tilt

Auto rebuild/reload services in-cluster on change.

🦴 Skaffold

Build & deploy workflow automation.

🛰️ Telepresence

Connect local dev to remote cluster.

🪞 mirrord

Mirror remote traffic into local process.

🎛️ Tilt in action

```
include('../../06-demo-application/api-golang/Tiltfile')
include('../../06-demo-application/api-node/Tiltfile')
include('../../06-demo-application/client-react/Tiltfile')
include('../../06-demo-application/load-generator-python/Tiltfile')
```

Each service has a `Tiltfile`; changes rebuild/reload automatically in the cluster.

10.2 Secrets management

Config lives safely in Git — but credentials **can't**. Common approaches:

🤮 1 · Manually create/update secrets out of band

No setup, but quickly unmanageable — eventually painful.

✅ 2 · External Secrets Operator (ESO) — used here

Mirrors secrets from an external provider (Google Cloud Secret Manager, 1Password) into Kubernetes Secrets. Sensitive data stays in a purpose-built tool; only non-sensitive references live in Git.

🔒 3 · Sealed Secrets / SOPS

Encrypt secrets so the **encrypted** form can be safely committed; decrypt in-cluster for use.

🚫 4 · Skip k8s secrets entirely

Load directly from Vault, AWS Secrets Manager, or Doppler.

10.3 ESO in practice

🏬 ClusterSecretStore + ExternalSecret (GCP)

```
kind: ClusterSecretStore
metadata: {name: gcp-store}
spec:
  provider:
    gcpsm: {projectID: kubernetes-course-424917}
---
kind: ExternalSecret
metadata: {name: example, namespace: external-secrets}
spec:
  refreshInterval: 1h
  secretStoreRef: {kind: ClusterSecretStore, name: gcp-store}
  target: {name: external-secrets-example-k8s, creationPolicy: Owner}
  data:
    - secretKey: key-k8s
      remoteRef: {key: external-secrets-example-gcp}
```

An `ExternalSecret` becomes a real k8s Secret, refreshed hourly.

🪪 Workload Identity

A k8s ServiceAccount is annotated with the GCP service account (`iam.gke.io/gcp-service-account`) so the operator can access Secret Manager without static keys.

⎈ Install

```
helm repo add external-secrets https://charts.external-secrets.io
helm upgrade --install external-secrets external-secrets/external-secrets \
  -n external-secrets --values values.yaml --create-namespace --version 0.9.19
```

🌟 Key Takeaways — Chapter 10

- Self-service inner loop tools: **Tilt, Skaffold, Telepresence, mirrord**.

- Tilt auto rebuilds/reloads services on change via per-service `Tiltfile`s.
- Never store plaintext secrets in Git — four approaches to solve it.
- **ESO** mirrors external provider secrets → k8s Secrets; references (not values) live in Git.
- Workload Identity avoids long-lived cloud credential keys.

✅ Quick-Review Checklist

- Four inner-loop tools
- What Tilt automates
- Four secrets approaches
- ESO: ClusterSecretStore vs ExternalSecret
- Workload Identity purpose

📝 Practice Questions

Q1. **What problem does Tilt solve?**
→ Manual image rebuilds — it rebuilds/reloads changed services in-cluster automatically.

Q2. **Name two other inner-loop tools.**
→ Skaffold, Telepresence (also mirrord).

Q3. **What does ESO do?**
→ Mirrors secrets from an external provider into Kubernetes Secrets.

Q4. **Difference between ClusterSecretStore and ExternalSecret?**
→ ClusterSecretStore defines the provider connection; ExternalSecret requests a specific secret and creates a k8s Secret.

Q5. **How do Sealed Secrets / SOPS help?**
→ They encrypt secrets so the encrypted form can safely live in version control.

Q6. **What is Workload Identity for?**
→ Letting a k8s ServiceAccount assume a cloud IAM identity without static keys.

Kubernetes · Part 11

11 Debugging Kubernetes

When things go wrong: a systematic flow from symptom to root cause.

🧪 The exercise

The app is Google's **microservices-demo**, but with a few **intentional breaks** baked in. Find and fix them yourself *before* peeking at the diff/PR.

11.1 The debugging flow

🗺️ LearnK8s troubleshooting flowchart

A great structure: start from the symptom (can't reach the app?) and walk outward — Ingress → Service → Endpoints → Pod → Container.

1 · Is the app reachable? → **2 · Ingress / Route ok?** → **3 · Service has endpoints?** →
4 · Pod Running & Ready? → **5 · Container logs / config**

11.2 Common symptoms → causes

Symptom	Likely cause	First look at
`ImagePullBackOff`	Bad image name/tag, missing pull secret	`k describe pod`, `imagePullSecrets`
`CrashLoopBackOff`	App crashes on start / bad config	`k logs`, env vars, probes
`Pending`	Unschedulable: resources, taints, PVC	`k describe pod` events
Service unreachable	Selector ↔ pod label mismatch	`k get endpoints`
Readiness never passes	Wrong probe path/port	probe config vs container
Wrong version deployed	Tag/stale cache, `imagePullPolicy`	Deployment image + rollout

11.3 The debugging toolkit

🔍 Inspect

```
kubectl get pods -o wide
kubectl describe pod <pod>      # events!
kubectl logs <pod> [-c co] [-f] [--
previous]
kubectl get events --sort-
by=.metadata.creationTimestamp
kubectl get endpoints <svc>     # empty
⇒ label/port mismatch
```

🩺 Interact

```
kubectl exec -it <pod> -- sh
kubectl port-forward svc/<svc> 8080:80
kubectl run tmp --rm -it --
image=nicolaka/netshoot -- bash
  # test DNS / connectivity from inside
```

🧭 Debugging mindset

- Work **outside-in** (user-facing symptom → inward) or **inside-out** (logs → outward) consistently.
- `describe` **events** often name the cause outright.
- Empty `Endpoints` almost always = label/selector or port mismatch.
- Change **one** thing at a time and re-check.

🌟 Key Takeaways — Chapter 11

- Debug **systematically** using a flow (Ingress → Service → Endpoints → Pod → Container).
- Know the big statuses: `ImagePullBackOff`, `CrashLoopBackOff`, `Pending`.
- `describe` events + `logs` + `get endpoints` solve most issues.
- Empty endpoints = selector/label/port mismatch.
- Use `exec`, `port-forward`, and a netshoot pod to test from inside.

✅ Quick-Review Checklist

- ✅ The debug flow order
- ✅ Meaning of the 3 big pod statuses
- ✅ Empty endpoints cause
- ✅ Commands: describe/logs/events/exec
- ✅ How to test in-cluster networking

📝 Practice Questions

Q1. A Service returns nothing. What's the first check?
→ `kubectl get endpoints` — empty means selector/label/port mismatch.

Q2. Pod stuck `ImagePullBackOff` — likely causes?
→ Wrong image name/tag or missing `imagePullSecrets`.

Q3. Pod is `CrashLoopBackOff`. Where to look?
→ `kubectl logs` (and `--previous`), plus env/config and probes.

Q4. A pod is `Pending`. What does that mean?
→ It's unschedulable — resources, taints/tolerations, or an unbound PVC.

Q5. Which command surfaces scheduling/probe messages?
→ `kubectl describe pod` (its Events section).

Q6. How do you test connectivity from inside the cluster?
→ Run a netshoot pod / `exec` into a pod, or `port-forward` a service.

Kubernetes · Part 12

12 Deploying to Multiple Environments

One set of manifests, many environments: Kustomize overlays, Helm values, Kluctl — and friends.

Same app, different targets (staging vs production). We want to vary a few things — **image tag**, **replicas**, **hostnames/DNS postfix** — *without* copying all the YAML.

12.1 Kustomize — the overlay model

🧩 Bases + overlays

Base YAML is merged with environment-specific **overlays** (patches). Built directly into `kubectl` (no extra tooling) — a good starting point, though **verbose** for complex patching.

📚 base/kustomization.yaml

```
kind: Kustomization
namespace: demo-app
resources:
  - api-golang
  - api-node
  - client-react
  - common
  - load-generator-python
```

🏭 production/api-node/kustomization.yaml

```
kind: Kustomization
namespace: demo-app
resources: ["../../base/api-node/"]
patches:
  - path: ./patches/Deployment.yaml
  - path: ./patches/IngressRoute.replace-
host.yaml
    target: {group: traefik.containo.us,
kind: IngressRoute,
            name: api-node, namespace:
demo-app, version: v1alpha1}
```

🩹 patches/Deployment.yaml

```
kind: Deployment
metadata: {name: api-node, namespace:
demo-app}
spec:
  template:
    spec:
      containers:
        - name: api-node
          image: sidpalas/…-api-
node:PRODUCTION_VERSION
```

🩹 patches/IngressRoute.replace-host.yaml

```
- op: replace
  path: /spec/routes/0/match
  value: "Host(`kubernetes-
course.devopsdirective.com`)
          && PathPrefix(`/api/node`)"
```

12.2 Helm — values per environment

✸ Charts + values

A **chart** bundles resources behind a custom interface. Vary behaviour with per-environment values files. Very common, but needs a fair bit of boilerplate ("heavy" for some use cases).

📄 values.yaml

```
environmentPostfix: ""
replicas: 1
version: DEFAULT_VERSION
```

🧪 values.staging.yaml

```
environmentPostfix: "-staging"
version: STAGING_VERSION
```

🚀 values.production.yaml

```
replicas: 2
version: PRODUCTION_VERSION
```

12.3 Kluctl — hybrid of both

🦎 Templating + diff + GitOps

Takes the good parts of Helm and Kustomize. Adds **templating**, a **diff** of current vs desired state, and a **GitOps controller**. The full example deploys the app (+ 3rd-party deps) to **staging** and **production** — and is reused in Part 14 (CI/CD).

🧱 kluctl/deployment.yaml

```
vars:
  - file: config/{{ args.environment }}.yaml
deployments:
  - path: namespaces
  - barrier: true
  - include: third-party
  - barrier: true        # postgres depends on cloudnative-pg
  - include: services
commonLabels:
  devopsdirective.com/course: "kubernetes-course"
```

12.4 Not shown (but worth knowing)

Tool	Idea
Timoni	Uses **CUE** for type safety/validation; config shipped as a single OCI artifact; steeper curve.
CDK8s	Define resources in **general-purpose languages** (TS, Python, Java, Go) → dynamic generation & testing.
Pulumi	Also define k8s resources in real languages; great if already using Pulumi for IaC.

🌟 Key Takeaways — Chapter 12

- Multiple environments = vary a few values (tag, replicas, host), reuse everything else.
- **Kustomize**: base + overlays, built into `kubectl`; verbose for complex patches.
- **Helm**: charts + per-environment `values` files; common but boilerplate-heavy.
- **Kluctl**: hybrid of the two + diff + GitOps controller; reused in CI/CD.
- Alternatives: **Timoni** (CUE), **CDK8s/Pulumi** (real languages).

✅ Quick-Review Checklist

- ✅ Base vs overlay
- ✅ Kustomize patch types shown
- ✅ How Helm varies per env
- ✅ What makes Kluctl a hybrid
- ✅ Timoni/CDK8s/Pulumi distinctives

📝 Practice Questions

Q1. **What does Kustomize's overlay model do?**
→ Merges environment-specific patches onto shared base YAML.

Q2. **Kustomize advantage and drawback?**
→ Built into kubectl (no extra tooling); verbose/cumbersome for certain patches.

Q3. **How does Helm vary per environment?**
→ Separate `values.staging.yaml` / `values.production.yaml` files override the chart's defaults.

Q4. **What makes Kluctl a "hybrid"?**
→ Helm-like templating + Kustomize-like composition, plus diff output and a GitOps controller.

Q5. **Which tool uses CUE and OCI artifacts?**
→ Timoni.

Q6. **What do CDK8s and Pulumi have in common?**
→ Defining k8s resources in general-purpose programming languages.

Kubernetes · Part 13

13 Cluster & Node Upgrades

Keep the control plane and workers current — safely, without dropping workloads.

🛡️ Why upgrade?

1. Patch **security vulnerabilities**.
2. Use **new features** as they're released.

☁️ It depends on your provider

Managed clusters differ: **GKE** can handle most upgrades transparently (release channels: **stable / regular / rapid**). **AWS EKS** needs a more hands-on approach. Here we do it manually.

13.1 The upgrade process

1

Check for deprecated APIs — address anything an upgrade would remove *before* upgrading (`kubent`).

2

Update the control plane — it can be ahead of workers but never behind, so it goes first.

3

Create a new nodepool — safer than in-place; new nodes become fully operational before shifting load.

4

Drain & cordon the old nodes — use scheduling to shift pods onto the new nodes.

5

Delete the old nodepool — once everything runs on the new nodes.

🔑 Golden rule

The **control plane can be ahead of worker nodes, but never behind**. Upgrade the control plane first.

13.2 The commands (GKE)

🧹 1 · Detect deprecated APIs

```
kubent          # Kube No Trouble — finds removed/removed-soon APIs
```

📋 2 · See available versions / pick a channel

```
gcloud container get-server-config --format "yaml(channels)"
gcloud container clusters update $CLUSTER \
  --release-channel rapid
```

🛫 3 · Upgrade the control plane

```
gcloud container clusters upgrade $CLUSTER \
  --zone $ZONE --master \
  --cluster-version 1.30.1-gke.1329003
```

🧱 4 · Create the new nodepool

```
gcloud container node-pools create updated-node-pool \
  --cluster $CLUSTER --zone $ZONE \
  --machine-type e2-standard-2 --num-nodes 2 \
  --workload-metadata=GKE_METADATA
```

🚚 5 · Cordon & drain, then delete old pool

```
nodes=$(kubectl get nodes -l cloud.google.com/gke-nodepool=default-pool -o name)
for node in $nodes; do
  kubectl cordon $node
  kubectl drain $node --ignore-daemonsets --delete-emptydir-data --force
done
gcloud container node-pools delete default-pool --cluster $CLUSTER --zone $ZONE
```

⛔ cordon

Marks a node **unschedulable** — no new pods land on it.

🚰 drain

Evicts existing pods so they reschedule elsewhere.

🌟 Key Takeaways — Chapter 13

- Upgrade for **security patches** and **new features**.
- Managed (GKE release channels) vs hands-on (EKS); here we do it manually.

- 5-step process: deprecations → control plane → new nodepool → cordon/drain → delete old.
- Control plane **ahead** of workers, never behind.
- New nodepool is safer than in-place: roll back by shifting workloads back.
- `kubent` finds deprecated API usage.

✅ Quick-Review Checklist

- ✅ Two reasons to upgrade
- ✅ The 5-step order
- ✅ Control plane vs workers ordering rule
- ✅ cordon vs drain
- ✅ Tool for deprecated APIs

📝 Practice Questions

Q1. **Why upgrade your cluster?**
→ Security patches and access to new features.

Q2. **What must be checked before upgrading?**
→ Usage of deprecated APIs that the upgrade removes (use `kubent`).

Q3. **Why update the control plane first?**
→ It can be ahead of workers but never behind.

Q4. **Why create a new nodepool instead of in-place?**
→ New nodes are fully ready before shifting load; you can roll back by shifting workloads to the old nodes.

Q5. **Difference between cordon and drain?**
→ Cordon stops new scheduling; drain evicts existing pods.

Q6. **What release channels does GKE offer?**
→ stable, regular, rapid.

Kubernetes · Part 14

14 CI/CD for Kubernetes

From image build to GitOps: GitHub Actions, image-tag updates, and continuous reconciliation.

14.1 The four CI/CD components

1️⃣ Build images

Compile/build the container image.

2️⃣ Push to registry

Publish the tagged image.

3️⃣ Update manifests

Bump k8s manifests to the new image tags.

4️⃣ Apply to cluster(s)

Roll the updated manifests out.

🧹 CI also does...

Linting, tests, etc. — not Kubernetes-specific, so omitted here for brevity.

14.2 Reference CI (GitHub Actions)

🐙 Image tag generation + manifest update

```
git describe --tags --always --first-parent \
  --match "[0-9]*.[0-9]*.[0-9]*"      # derive version tag
```

A brittle string-replacement step updates image tags in `12- .../kluctl` *by searching for marker comments like* `# STAGING_IMAGE_TAG` / `# PRODUCTION_IMAGE_TAG`.

⚠️ More robust alternatives

Use **Flux image automation** or **Renovate** (regex manager) to detect new tags and update manifests instead of brittle string replacement.

14.3 Keeping cluster state in sync

Approach	Pros	Cons
Manual `kubectl apply`	Simple	Someone forgets, or hits the wrong cluster
Automated apply from pipeline	Removes human error	Cluster creds in CI; **no drift reconciliation**
GitOps (pull & reconcile)	Strongest security boundary; drift detected/eliminated	Requires a controller

14.4 GitOps with Kluctl

🔄 Kluctl GitOps controller

The controller continuously pulls config from Git and reconciles the cluster. A `KluctlDeployment` points at a repo path and target (staging/production).

📄 KluctlDeployment (staging)

```
kind: KluctlDeployment
metadata: {name: demo-app, namespace: kluctl-gitops}
spec:
  interval: 5m
  source:
    git:
      url: https://github.com/sidalas/devops-directive-kubernetes-course.git
      path: 12-deploying-to-multiple-environments/kluctl
  target: staging
  context: default
  prune: true     # delete orphaned resources
  delete: true
```

🧭 Self-managing GitOps repo

```
# .kluctl.yaml — targets + discriminator
targets:
  - {name: staging,    args: {cluster_name: staging},    context: devops-directive-kubernetes-
course}
  - {name: production, args: {cluster_name: production}, context: gke_…}
discriminator: gitops-{{ args.cluster_name | slugify }}
# clusters/deployment.yaml installs the controller + per-cluster items
```

*The GitOps deployment even manages **itself** — bootstrapping the controller and its own* `KluctlDeployment`.

ArgoCD

Popular pull-based GitOps tool.

FluxCD

Popular GitOps toolset (incl. image automation).

Key Takeaways — Chapter 14

- CI/CD for k8s = **build** → **push** → **update tags** → **apply**.
- Version tags via `git describe`; manifest updates via marker comments (fragile) or Flux/Renovate (robust).
- Three sync strategies: manual, pipeline apply, **GitOps** (best: drift reconciliation).
- **GitOps** = cluster continuously pulls from Git; strongest security boundary.
- Kluctl controller + `KluctlDeployment` (interval, prune, delete); alternatives ArgoCD & FluxCD.
- Kluctl can even deploy/manage the GitOps controller itself.

Quick-Review Checklist

- [x] The four CI/CD components
- [x] How version tags are derived
- [x] Three cluster-sync strategies & trade-offs
- [x] What "GitOps" means
- [x] KluctlDeployment fields (interval/prune/delete)
- [x] ArgoCD & FluxCD

Practice Questions

Q1. **What are the four CI/CD components for k8s?**
→ Build images, push to registry, update manifests, apply to cluster(s).

Q2. **How is the version tag generated in the reference flow?**
→ `git describe --tags … --match "[0-9]*.[0-9]*.[0-9]*"`.

Q3. **Why is string-replacement tag updating fragile?**
→ It relies on comment markers/regex; Flux image automation or Renovate is more robust.

Q4. **Trade-off of automated (non-GitOps) apply?**
→ Needs cluster creds in CI and doesn't reconcile drift.

Q5. **What is GitOps?**
→ The cluster continuously pulls config from version control and reconciles state, eliminating drift.

Q6. **Name two popular GitOps tools besides Kluctl.**
→ ArgoCD and FluxCD.

A

Cheat Sheet · Part 1

Docker Cheat Sheet

Every command & concept from the Docker course, condensed.

Core concepts

🐳 What is a container?

A runnable instance of an image — an isolated process (namespaces + cgroups) with its own filesystem.

🥞 Image vs Container vs Layer

Image = read-only template (stack of layers). **Container** = image + a writable layer. **Layer** = cached step.

🧱 Linux primitives

namespaces = isolation · **cgroups** = limits · **union FS** = layered images.

🏗️ Architecture

Docker daemon (`dockerd`) + client + registry. Builds by **containerd/runc**.

Images & building

🔨 Build & tag

```
docker build -t myapp:1.0 .
docker build -t myapp:1.0 -f
Dockerfile.dev .
docker tag myapp:1.0 user/myapp:1.0
```

📦 Inspect & manage

```
docker images
docker image inspect myapp:1.0
docker history myapp:1.0
docker rmi myapp:1.0
docker image prune -a
```

🏆 Dockerfile best practices

Order layers **least → most frequently changing**; combine `RUN` s; **multi-stage** builds to shrink final image; use `.dockerignore` ; pin base image tags; run as non-root; use `COPY` not `ADD` ; add a `HEALTHCHECK` .

🧰 Common instructions

```
FROM node:20-alpine        # base image (stage)
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
USER node
CMD ["node", "server.js"]
# multi-stage: FROM builder AS build … FROM gcr.io/distroless/nodejs
```

Running containers

▶️ docker run — the important flags

```
docker run -d --name web -p 8080:80 \
  -v mydata:/var/lib/data \           # named volume
  -v $PWD/src:/app:ro \               # bind mount (read-only)
  -e NODE_ENV=production \            # env var
  --env-file .env \
  --network mynet \
  --restart unless-stopped \
  --rm -it \
  myapp:1.0
```

🔍 Inspect

```
docker ps -a
docker logs -f web
docker inspect web
docker stats
docker top web
```

🕹️ Control

```
docker stop web
docker start web
docker restart web
docker exec -it web sh
docker cp web:/file .
docker rm -f web
```

Compose, volumes, networks, registries

Docker Compose

```
docker compose up -d
docker compose down -v
docker compose logs -f api
docker compose build
docker compose ps
```

Services, networks & volumes declared in `compose.yaml`.

Volumes

```
docker volume create mydata
docker volume ls
docker volume inspect mydata
docker volume rm mydata
```

Networks

```
docker network create mynet
docker network ls
docker network connect mynet web
docker network inspect mynet
```

Registry

```
docker login
docker pull nginx:1.27
docker push user/myapp:1.0
docker logout
```

Security & maintenance

Harden images

- Minimal base (alpine/distroless), non-root `USER`.
- Pin & scan images (Trivy/Grype); sign (cosign).
- Read-only root FS, drop capabilities, no secrets in layers.

Cleanup / disk

```
docker system df
docker system prune -a --volumes
docker image prune -f
docker container prune -f
```

Remember

-v named volume persists data · **bind mount** maps a host path · **-p host:container** publishes a port · **--rm** auto-deletes on exit.

Portable question	Answer
Difference between image & container	Image is a read-only template; a container is a running instance with a writable layer.
What does `-p 8080:80` do?	Publishes host port 8080 → container port 80.
Why multi-stage builds?	Keep build tooling out of the final image → smaller & more secure.
Named volume vs bind mount?	Volume is Docker-managed; bind mount points at a host path.
What is `compose` for?	Define & run multi-container apps declaratively.

B

Cheat Sheet · Part 2

Kubernetes Cheat Sheet

kubectl, YAML anatomy & every resource type from the course.

Control plane & nodes

🧠 Control plane

kube-apiserver (API) · **etcd** (state) · **scheduler** · **controller-manager**.

🖥️ Data plane (node)

kubelet · **kube-proxy** · **container runtime** (containerd). **CoreDNS** = cluster DNS.

🔁 The reconciliation loop

Declare **desired state** (YAML) → controllers continuously **reconcile** actual state toward it.

YAML anatomy (every object)

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: api, namespace: demo-app, labels: {...}}
spec:                       # desired state → resource-specific
status: {...}               # actual state → set by the controller
```

kubectl — the essentials

🔎 Read

```
k get pods -A -o wide
k get all -n demo-app
k describe pod <pod>       # events!
k logs <pod> -c <c> -f --previous
k get events --sort-by=.lastTimestamp
k get pods -l app=api -w
```

✏️ Change & interact

```
k apply -f manifest.yaml
k apply -f ./dir/             # or kustomize dir
k delete -f manifest.yaml
k edit deploy/api
k scale deploy/api --replicas=4
k exec -it <pod> -- sh
k port-forward svc/api 8080:3000
k rollout status deploy/api
k rollout undo deploy/api
```

🗂️ Contexts & namespaces

```
k config get-contexts
k config use-context dev
k create ns demo-app
k config set-context --current --
namespace=demo-app
kubens / kubectx
```

🧨 Debug helpers

```
k get pods --field-selector
status.phase≠Running
k get endpoints <svc>       # empty ⇒
label/port mismatch
k run tmp --rm -it --
image=nicolaka/netshoot -- bash
k api-resources
```

Resource types at a glance

Kind	What it does	Key fields
Namespace	Virtual cluster / scoping	`metadata.name`
Pod	Smallest deployable unit (1+ containers)	`spec.containers`
ReplicaSet	Keeps N pod replicas running	`replicas`, `selector`
Deployment	Declarative rollout & revision of ReplicaSets	`template`, strategy
Service	Stable LB/DNS to pods	`selector`, `ports` (ClusterIP/NodePort/LB)
Ingress	HTTP routing into cluster	rules, `ingressClassName`
Job / CronJob	Run-to-completion / scheduled	`template`, `schedule`
DaemonSet	One pod per node	`template`
StatefulSet	Stable identity + ordered storage	`serviceName`, `volumeClaimTemplates`
ConfigMap / Secret	Config / sensitive data	`data` / `stringData`
PV / PVC / StorageClass	Persistent storage provisioning	`accessModes`, `storageClassName`

Mini-manifests

🚀 Deployment + Service

```
kind: Deployment
spec:
  replicas: 2
  selector: {matchLabels: {app: api}}
  template:
    metadata: {labels: {app: api}}
    spec:
      containers:
        - {name: api, image:
user/api:1.0,
           ports: [{containerPort:
3000}],
           readinessProbe: {httpGet:
{path: /ping, port: 3000}}}
---
kind: Service
spec:
  selector: {app: api}
  ports: [{port: 80, targetPort: 3000}]
```

🔐 ConfigMap + Secret

```
kind: ConfigMap
data: {APP_MODE: "prod"}
---
kind: Secret
type: Opaque
stringData: {DATABASE_URL:
"postgres://…"}
# consume:
envFrom:
  - configMapRef: {name: app-config}
  - secretRef:    {name: db-secret}
env:
  - {name: X, valueFrom: {secretKeyRef:
      {name: db-secret, key:
DATABASE_URL}}}
```

Probes, resources & security

❤️ Probes

liveness restarts · **readiness** gates traffic · **startup** for slow boot.

📊 Resources

`requests` = scheduling guarantee · `limits` = hard cap.

🔒 securityContext

non-root, `allowPrivilegeEscalation:false`, `readOnlyRootFilesystem`, seccomp.

Helm, Kustomize & GitOps

⎈ Helm

```
helm repo add bitnami
https://charts.bitnami.com/bitnami
helm install pg bitnami/postgresql -f
values.yaml
helm upgrade --install pg
bitnami/postgresql
helm list / helm rollback pg 1 / helm
uninstall pg
```

🧩 Kustomize

```
kustomize build overlays/production
k apply -k overlays/production
# base/ + overlays/ patches
```

🔄 GitOps

Cluster pulls config from Git and reconciles continuously → drift eliminated. Tools: **Kluctl**, **ArgoCD**, **FluxCD**.

📌 Remember

Debug outside-in: Ingress → Service → `Endpoints` → Pod → Container. Empty endpoints = label/port mismatch.

Cheat Sheet · Part 3

Master Cheat Sheet

The entire Docker + Kubernetes course condensed to one spread.

The big picture

Source code → Dockerfile → Image → Registry → K8s manifests → Cluster → GitOps reconcile

Docker in one table

Concept	One-liner
Container	Isolated process: namespaces + cgroups + union FS.
Image / Layer	Layered read-only template; each Dockerfile step is a cached layer.
Build	`docker build -t app:1.0 .` — multi-stage to slim the result.
Run	`docker run -d -p 8080:80 -v vol:/data --name web app:1.0`
Compose	`docker compose up -d` — multi-container apps from YAML.
Registry	`docker push user/app:1.0` — Docker Hub / GHCR / ECR.
Security	Minimal base, non-root, scan & sign, no secrets in layers.

Kubernetes in one table

Layer	Key pieces
Control plane	apiserver · etcd · scheduler · controller-manager
Node	kubelet · kube-proxy · containerd · CoreDNS
Workloads	Pod · ReplicaSet · Deployment · StatefulSet · DaemonSet · Job/CronJob
Networking	Service (ClusterIP/NodePort/LB) · Ingress/Gateway · NetworkPolicy
Config	ConfigMap · Secret · ServiceAccount · RBAC
Storage	PersistentVolume · PersistentVolumeClaim · StorageClass
Packaging	Helm · Kustomize · Kluctl

Mental models & rules of thumb

🔁 Declarative + reconciliation

Everything is desired state; controllers close the loop. Extends to CRDs/Operators.

🧱 Labels & selectors glue it together

Services, Deployments & ReplicaSets find pods via **label selectors**. Mismatch = broken.

🛡️ Least privilege everywhere

Non-root containers, dropped caps, RBAC, no plaintext secrets, signed images.

🔍 Debug systematically

`describe` events → `logs` → `get endpoints` → exec/port-forward.

The delivery pipeline (CI/CD)

1️⃣ Build

Image built in CI.

2️⃣ Push

Tagged & pushed to registry.

3️⃣ Update

Manifests get the new tag (Renovate/Flux).

4️⃣ Deploy

GitOps reconciles the cluster.

Security checklist (full stack)

🐋 Image / build

- Minimal, pinned base images; multi-stage.
- Non-root user; read-only root FS.
- Scan (Trivy) & sign (cosign); SBOM.
- No secrets baked into layers.

☸️ Cluster / runtime

- RBAC least-privilege; ServiceAccounts per workload.
- NetworkPolicies; Pod Security standards.
- Secrets from external store (ESO), not Git.
- Requests/limits; probes; upgrade regularly.

Rapid-fire recall

📝 12 Questions to self-test

1. **Image vs container?**
→ Template vs running instance (writable layer).
2. **Why multi-stage builds?**
→ Smaller, more secure final image.
3. **What glues k8s pods to services?**
→ Label selectors.
4. **Deployment vs StatefulSet?**
→ Stateless rolling vs stable identity + storage.
5. **ConfigMap vs Secret?**
→ Non-sensitive vs sensitive (base64/opaque).
7. **What is GitOps?**
→ Cluster pulls & reconciles config from Git; no drift.
8. **First step of a cluster upgrade?**
→ Check for deprecated APIs (kubent).
9. **Empty Service endpoints mean?**
→ Selector/label or port mismatch.
10. **Helm vs Kustomize?**
→ Templating + releases vs base/overlay patching.
11. **How to avoid static cloud creds?**
→ Workload Identity / IAM roles for ServiceAccounts.

6. What are CRDs for?
→ Extending the API with custom resource types.

12. Kernel features behind containers?
→ namespaces, cgroups, union/overlay FS.

🎓 You made it!

Docker builds the artifact. Kubernetes runs it, at scale, declaratively. CI/CD ships it. Security applies at every layer.